

Documento de Especificação de Projeto Integrador

MeatShop - Açougues Online

SUMÁRIO

1. PARTE 00 - HISTÓRICO DE VERSÃO DO PROJETO	4
2. PARTE 01 – IDENTIFICAÇÃO DO PRODUTO, EQUIPE E REPOSITÓRIO	6
3. 1 IDENTIFICAÇÃO DO PRODUTO	6
4. 2 DESCRIÇÃO GERAL DO PRODUTO	6
5. 2.1 Resumo do Negócio	6
6. 2.2 Problemas Identificados	7
7. 3 EQUIPE DO PROJETO	8
8. 4 REPOSITÓRIOS E ARTEFATOS DO PROJETO	9
9. PARTE 02 – PLANEJAMENTO DO SEMESTRE	9
10. 5 SITUAÇÃO ATUAL DO PROJETO	9
11. 6 OBJETIVOS DO SEMESTRE	10
12. 7 CRONOGRAMA DO SEMESTRE	11
13. PARTE 03 – ENTENDIMENTO DO PÚBLICO, DO NEGÓCIO E LEVANTAMENTO INICIAL DE REQUISITOS	12
14. 8 PÚBLICO-ALVO DO PRODUTO	12
15. 9 CONTEXTO DE NEGÓCIO	12
16. 10 SOLUÇÕES EXISTENTES	13
17. 11 PESQUISA COM USUÁRIOS	14
18. 12 RESULTADOS DE PESQUISA COM USUÁRIO	14
19. 13 REGRAS DE NEGÓCIO	17
20. 14 REQUISITOS FUNCIONAIS	22
21. 15 REQUISITOS NÃO FUNCIONAIS	26
22. 16 RASTREABILIDADE DE REQUISITOS	30
23. 17 FUNDAMENTAÇÃO TEÓRICA	31
24. 17.1 Transformação digital no varejo e oportunidade de negócio	31
25. 17.2 E-commerce, delivery especializado e valor agregado ao cliente	32
26. 17.3 Experiência do usuário, usabilidade e acessibilidade	32
27. 17.4 Engenharia de requisitos e rastreabilidade	33
28. 17.5 Arquitetura de software em camadas	34
29. 17.6 Segurança da informação e proteção de dados	34

30. 17.7 Síntese da fundamentação aplicada ao MeatShop	35
31. PARTE 04 – DESIGN, MODELAGEM, DADOS E SOLUÇÃO TECNOLÓGICA	35
32. 18 DESIGN DA SOLUÇÃO E EXPERIÊNCIA DO USUÁRIO	35
33. 18.1 Fluxo de Navegação	35
34. 18.2 Protótipos do Sistema	36
35. 19 GESTÃO DO PROJETO DE MODELAGEM DO SISTEMA	36
36. 19.1 Organização do Projeto (Abordagem Ágil)	36
37. 19.2 Modelagem e Diagramas do Sistema	36
38. 20 DADOS E MODELAGEM DA INFORMAÇÃO	37
39. 20.1 Modelo de Dados	37
40. 20.2 Estrutura e Manipulação de Dados	38
41. 21 SOLUÇÃO TECNOLÓGICA E EXPERIMENTAÇÃO TÉCNICA INICIAL	39
42. 21.1 Escolhas Tecnológicas	39
43. 21.2 Experimentação Técnica e Primeiros Testes	40
44. PARTE 05 – ARQUITETURA, TECNOLOGIA E IMPLEMENTAÇÃO EVOLUTIVA DO SISTEMA	43
45. 22 ARQUITETURA GERAL DO SISTEMA	43
46. 23 TECNOLOGIAS E FERRAMENTAS DO PROJETO	45
47. 24 IMPLEMENTAÇÃO POR CAMADAS DO SISTEMA	47
48. 24.1 Frontend	47
49. 24.2 Backend	49
50. 24.3 Integração	49
51. 25 ORGANIZAÇÃO DO CÓDIGO E REPOSITÓRIO	52
52. PARTE 06 – QUALIDADE, SEGURANÇA, AVALIAÇÃO E EXTENSÕES DO PROJETO	54
53. 27 QUALIDADE DO SOFTWARE E TESTES	54
54. 28 SEGURANÇA DA INFORMAÇÃO	60
55. 29 EXTENSÕES E TECNOLOGIAS COMPLEMENTARES	
56. 30 AVALIAÇÃO GERAL DO PROJETO INTEGRADOR	
57. 31 REFERÊNCIAS	62

PARTE 00 - HISTÓRICO DE VERSÃO DO PROJETO

Histórico de versão Web

Versão	Período	Fase	Data	Descrição
Meatshop: 1.0.0	7º	01	08/03/2026	Frontend, Backend e banco integrados de forma completa bem como funcionalidades do Web como controle do estoque, gerenciamento de pedidos e geração de gráficos da receita mensal funcionais.
Meatshop: 1.1.0	7º	02	26/03/2026	Integração do Back-end com a api do mercado pago permitindo o cliente realizar os pagamentos pelo mercado pago.
Meatshop: 1.2.0	7º	02	06/04/2026	Inserção da tela de configurações do perfil do usuário para modificação de informações do usuário e troca de login do sistema.

Histórico de versão Mobile

Versão	Período	Fase	Data	Descrição
Meatshop: 1.0.0	7º	01	11/03/2026	Estrutura inicial do mobile, criação das telas(Splash, Boas-vindas, Login, Seleção de Tipo de Cadastro, Cadastro de usuário, Alteração de Senha) e validação de força de senha em tempo real.
Meatshop: 1.1.0	7º	02	09/04/2026	Criação de novas telas(listagem de açougues,tipos de cortes e acompanhamento de entrega) e implementação do fluxo completo de autenticação login com validação e logout.

Meatshop: 1.2.0	7º	02	15/04/2026	Inserção das telas(acompanhamento de entrega, detalhes do açougue e detalhes do produto), além do fluxo de chat com estabelecimentos que inclui as telas de conversa e listagem de chat.
Meatshop: 1.3.0	7º	03	22/04/2026	Fluxo de gerenciamento de endereços, tela de configurações da conta e fluxo de adição de cartões para pagamentos.
Meatshop: 1.4.0	7º	03	26/04/2026	Fluxo completo da conta do entregador, fluxo de entrega de duas etapas, integração de mapa interativo, rastreamento em tempo real do pedido, além de atualizações para fluxo de 2º tipo de usuário e funcionalidades do usuário entregador.
Meatshop: 1.5.0	7º	03	30/04/2026	Tela de gestão do entregado, tela de pagamento para pagamento online ou na entrega, além de atualizações em algumas telas para melhor visualização.
Meatshop: 1.6.0	7º	03	04/05/2026	Inserção da nova tela de seleção de endereço com modalidade de entrega, expansão do carrossel de promoções, e início da integração da IA de receitas do meatshop.

PARTE 01 – IDENTIFICAÇÃO DO PRODUTO, EQUIPE E REPOSITÓRIO

1 IDENTIFICAÇÃO DO PRODUTO

Nome do Produto:

MeatShop – Açougues online

Descrição do Produto:

O MeatShop é um site/aplicativo criado para açougues e seus clientes que permite aos açougues controlarem seus estoques, pedidos e vendas, já os clientes podem ver os produtos, fazer pedidos e acompanhar suas compras. O objetivo deste projeto é oferecer uma solução digital completa para o mercado de carne, com uma interface fácil de usar, que auxilia no gerenciamento dos açougues enquanto entrega uma experiência de compra simples e prática para os clientes.

2 DESCRIÇÃO GERAL DO PRODUTO

2.1 Resumo do Negócio

O meatshop foi pensado para facilitar a compra de carnes entre outros itens de açougue visto que normalmente para se obter estes produtos é necessário se deslocar até o estabelecimento já que são poucos os açougues que possuem entregadores desta forma o mercado de carnes para preparo e itens de açougue não tem uma presença muito forte do delivery que acaba sendo mais focado em comidas já prontas, dessa forma o cliente que necessita de uma carne para preparo seja para alguma confraternização ou porque esqueceu de tirá-la para descongelar, dessa forma para suprir esta dor do mercado o meatshop fará essa conexão do cliente com o açougue e facilitará a venda de carnes para preparo.

2.2 Problemas Identificados

Atendimento demorado: normalmente nos açougues conforme a clientela chega ela é atendida ou seja atendimento por filas em alguns casos possuem até sistema de senha para atendimento para por exemplo pessoas mais velhas terem prioridade porém ainda é um método de atendimento muito lento no geral.

Deslocamento: O cliente quase sempre tem que se deslocar ao açougue para ser atendido, afinal não existem muitos que tem um atendimento nem que seja por whatsapp dessa forma sempre é necessário ir ao açougue sem uma forma de atendimento online.

Controle dos pedidos: caso o açougue esteja com muitos clientes os açougueiros podem confundir pedidos, quantidades tipo de carne pela alta quantidade de clientes ou até mesmo caso o cliente realize um pedido pelo whatsapp e venha buscar pessoalmente o pedido pode vir a estar errado já que o atendimento é realizado por fala onde cliente pede o que deseja e o açougueiro entrega então tudo depende da memória dele que pela correria do trabalho pode gerar esquecimentos e confusões na hora de separar o pedido.

Item	Descrição
O problema de	Atendimento demorado que depende do próprio açougueiro ver como o cliente o que ele precisa e então realiza os cortes o que gera uma demora principalmente quando a muitos clientes no açougue
Afeta	Clientes dos açougues e os próprios açougueiros
Cujo impacto é	Demora no atendimento que gera insatisfação no cliente assim diminuindo as vendas do açougue.
Benefícios de uma solução seriam	Economia de tempo pelo cliente e pelo açougue que pode atender mais clientes por vez.

Item	Descrição
O problema de	Necessidade de se deslocar até o açougue, o que gasta o tempo do cliente.
Afeta	Clientes dos açougues
Cujo impacto é	O tempo gasto para se locomover até o açougue que corre o risco de ficar em filas ou não encontrar a carne que deseja.
Benefícios de uma solução seriam	Economia de tempo e praticidade ao cliente que não precisa se locomover até o estabelecimento.

Item	Descrição
O problema de	Ausência de um controle dos pedidos não tendo rastreio e nem registro o que pode ocasionar em erros nos pedidos(quantidade, tipo de carne etc.)
Afeta	Os açougues.
Cujo impacto é	Falta de gerenciamento fazendo com que o açougueiro se perca nos pedidos(caso haja um grande movimento de clientes) não sabendo quais atendidos, até misturando eles.
Benefícios de uma solução seriam	Controle dos pedidos permite ao açougue saber quais foram atendidos além de conseguir atendê-los corretamente, afinal o açougueiro não terá que memorizar os pedidos em casos de grande movimento.

3 EQUIPE DO PROJETO

Matrícula	Nome Completo	Função no Projeto
2313275	Geovanna Teresa Félix	Frontend + UX/UI
2310947	Rafael França Martins	Backend + Segurança + Infraestrutura
2310544	Samuel Elias Moraes Pereira	Documentação do APP + QA/Testes
2311066	Vinícius Rodrigues Martins	Mobile + Banco de Dados
1-11912	Éder José Almeida da Silva	Orientador Principal
11394	Talles Santos Faria Silva	Orientador Secundário

4 REPOSITÓRIOS E ARTEFATOS DO PROJETO

Tipo de Artefato	Link	Descrição
Repositório de código	https://github.com/geovannaffelix/meatshop https://github.com/VIN1RM/meatshop-mobile	1º Repositório - Frontend(web)/Backend da Aplicação 2º Repositório - Mobile
Protótipo das telas mobile	https://www.figma.com/design/bsnEodDFnhljAUC4FMmxXP/Meat-Shop?node-id=0-1&p=f&t=5liC8eL58oAKyX7V-0	Link do figma dos protótipos das telas mobile
Telas web	https://www.figma.com/design/xaTRgRIxZGSPydSWqmqsfD/Web?node-id=0-1&p=f	Link do figma dos protótipos telas do frontend
Chatbot de receitas	https://github.com/VIN1RM/meatshop-chatbot.git	Link da API de chatbot inteligente especializada em carnes

PARTE 02 – PLANEJAMENTO DO SEMESTRE

5 SITUAÇÃO ATUAL DO PROJETO

Neste período o projeto já com um desenvolvimento avançado onde já a aplicação web para o açougue já temos telas do front-end prontas de login, cadastro, pedidos, controle de estoque, receita prontas todas integradas com o back-end e o banco de dados tudo funcional e interativo.

Na aplicação web já é possível criar uma conta, logar, esqueci minha senha bem como a aba de pedidos pendentes e pedidos que saíram para a entrega, controle de estoque para visualização do açougue quanto a seus produtos em estoque e os relatórios da lucros e despesas do açougue gerando um resumo das receitas do mês. Apesar de ainda não estar hospedado, porém já está em discussão com orientador para resolvermos está pendência além disso as funcionalidades específicas para um usuário entregador que verão os pedidos disponíveis para entrega e escolherem qual eles querem entregar estão sendo planejadas para mobile assim como a produção da aplicação mobile para o cliente do açougue fazer seus pedidos, filtrar por preço, proximidade avaliações e até deixar pedidos

agendados para datas específicas.

Fase 02

Nesta segunda fase o projeto foi atualizado com as devidas proposta no objetivo e cronograma do projeto sendo a integração com api do mercado pago para que o pagamento possa ser realizado via mercado pago, além da adição da tela de configurações do perfil do usuário para alteração e inserção de informações do usuário. Além disso houve um avanço na aplicação mobile com a criação de diversas telas como login, cadastro, redefinição de senha, telas de listagem de açougues e cortes de carnes, acompanhamento de entrega e chat com açougue.

6 OBJETIVOS DO SEMESTRE

O objetivo neste semestre é desenvolver a aplicação mobile do projeto onde o usuário cliente poderá realizar seus pedidos e pesquisas nos açougues, também está sendo avaliada a criação de um usuário entregador que verá quais entregas estão disponíveis e aceitar a entrega que for melhor para ele além de também estamos de visando um sistema que mostrará a posição do entregador para o cliente saber aonde a sua entrega está e o com próxima ela está de ser entregue, a parte do cliente é de extrema importância afinal ela é a chave do nosso negócio é com ela que traremos a praticidade que é um dos diferenciais do projeto já o do entregador será mais um diferencial já que aplicativos concorrentes não tem o acompanhamento de onde o entregador se encontra além de facilitar para açougues que não possuem entregador essa parte mobile está sendo planejada para este momento pois a parte web já está concluído até o momento e agora poderemos parte para o mobile que é um área ainda não vista neste período do curso porém temos a intenção de adiantar para não haver uma sobrecarga de atividades para o nosso próximo semestre.

Fase 2 - 7º período

Após a entrega da fase 1 avançamos com o mobile criando diversos fluxos de telas e construindo o front do mobile a fim de integrá-lo ao backend e fazer toda a aplicação web e mobile estar funcional com o mesmo back e banco além de integrar testes para avaliar a integração dos mesmos sendo e também avaliar as funcionalidades separadamente neste momento realizaremos um teste com mocks para avaliar o cadastro de produtos e verificar alguns cenários que descumprem as regras do cadastro e um caso que cumpre as regras e mais para frente outros testes de mocks serão desenvolvidos para avaliar a gestão de pedidos e financeiro do açougue.

Fase 3 - 7º período

Após algumas dailys entre o grupo e professor que fazia a orientação do projeto decidimos integrar uma API de chatbot inteligente com a função de auxiliar os clientes do açougue a como preparar a carne respondendo perguntas sobre cortes de carne, receitas, técnicas de preparo e da cultura brasileira sendo projetada para ser especializada em carnes.

7 CRONOGRAMA DO SEMESTRE

Mês	Fase do projeto	Atividades principais
1º Mês	Sistema Web	Implementação das funcionalidades do sistema web para inserção e gerenciamento de informações do açougue, incluindo atualização de produtos, promoções, pedidos e status. além do gerenciamento do estoque e receita do mês.
2º Mês	Backend e Integrações	Desenvolvimento da lógica e das APIs do backend, incluindo integração com o Mercado Pago para pagamentos, comunicação entre os sistemas web e banco, além do gerenciamento de dados, autenticação e regras de negócio.
3º Mês	Aplicativo Mobile	Desenvolvimento do aplicativo mobile com criação dos fluxos de navegação do usuário, permitindo visualização de produtos, realização de pedidos, acompanhamento de status e recebimento de notificações.
		Desenvolvimento do mecanismo de

4° Mês	Sincronização de Sistemas	sincronização entre PostgreSQL e Firebase, utilizando gatilhos e processos automáticos para manter os dados atualizados entre os dois sistemas.
5° Mês	Testes e Ajustes	Realização de testes e ajustes contínuos em todos os sistemas (web, mobile e backend), garantindo o funcionamento integrado e a estabilidade da solução final.

PARTE 03 – ENTENDIMENTO DO PÚBLICO, DO NEGÓCIO E LEVANTAMENTO INICIAL DE REQUISITOS

8 PÚBLICO-ALVO DO PRODUTO

O público alvo do projeto são adultos e jovens adultos que tem num geral já trabalham e tem uma rotina muito puxada e atarefada onde eles não tem tempo pra ficar indo ao açougue comprar a carne da semana, ou caso você queira ir a uma confraternização e acaba esquecendo de comprar a carne antes e dessa forma precisa ir em cima da hora e o estabelecimento pode estar cheio ou até mesmo porque você esqueceu de tirar a carne para descongelar para o almoço em todos estes casos basta abrir o meatshop e solicitar um pedido para o açougue que mais convém ao usuário e então o entregador leva até o cliente caso queira que ele entregue no local da confraternização também será possível, e claro que também tem os açougues que não só terão um controle dos pedidos como também terão um sistema de gerenciamento de estoque e de receitas onde verão os gastos e lucros gerados pelo estabelecimento e claro outros estabelecimentos também se beneficiaram do nosso aplicativo como por exemplo uma hamburgueria pode deixar um pedido grande de carne agendado com açougue assim abastecendo seus estoques e facilitando o pedido de carnes para seu negócio.

9 CONTEXTO DE NEGÓCIO

O meatshop se encontra no ramo de delivery especificamente para açougues onde buscamos facilitar a compra de carnes enquanto ajudamos o açougue com outras demandas como estoque de produtos e relatório das receitas do açougue fazendo isso através de um sistema onde cliente precisa de uma carne para almoço então ele solicita um pedido, efetua o pagamento e então o açougue separa a carne entre outros produtos do pedido e por fim o entregador que aceitar a entrega realiza o transporte até o consumidor, este produto tem como objetivo principal é evitar a perda de tempo pelo consumidor que muitas vezes por estar em cima da hora acaba optando por pedir algo já pronto dessa forma nossa aplicação busca trazer praticidade ao cliente além de permitir que ele possa escolher o açougue que melhor lhe atenda com outros produtos também, maior alcance de vendas e atendimento para o açougue e até mesmo prover uma trabalho e uma renda para pessoas que se disponibilizam a realizar as entregas

10 SOLUÇÕES EXISTENTES

Primeiro existe o Ifood sendo o mais conhecido e consolidado no mercado porém na maioria das vezes sendo utilizado para pedir comidas prontas então mesmo possuindo uma área de pedidos de açougue entre outros o foco acaba sendo nos alimentos prontos e não para serem preparadas desta forma ele acaba atendendo a necessidade geral e não algo mais nichado como aqueles que querem preparar um almoço por exemplo ou fazer um churrasco afinal para que o cliente pediria algo para ser preparado quando o próprio ifood faz questão do produto pronto, dessa forma o nosso projeto atenderá este nicho mais específico além de ter outras vantagens como alguns serviços para o açougue, pedidos agendados para o cliente bem como acompanhamento em tempo real de onde o pedido se encontra.

Outra solução existente é o próprio 99food que assim como seu principal concorrente o ifood é mais focado em comidas prontas e mais tendo as mesmas desvantagens que seu rival apesar de oferecer taxas menores aos restaurantes.

O swift mercado de carnes é uma rede de açougues próprios que possuem várias lojas além de seu próprio aplicativo de entregas onde entregavam carnes frescas

e congeladas além de outros itens de açougue onde você poderia pedir para ser entregue ou ir buscar similar ao meatshop porém não apresentam um acompanhamento ou agendamento de pedidos e também não possuem mais uma filial em Anápolis limitando seu alcance para Brasília e Goiânia.

11 PESQUISA COM USUÁRIOS

Pergunta de Pesquisa: Como uma plataforma digital especializada em delivery de produtos de açougue pode reduzir o tempo gasto pelos consumidores em compras presenciais e aumentar a praticidade para pessoas com rotina intensa?

A pesquisa com usuários foi realizada com o objetivo de compreender os hábitos de compra relacionados a produtos de açougue e avaliar a aceitação de uma plataforma digital especializada nesse segmento. Para a coleta de dados foi utilizado um questionário estruturado por meio da plataforma Google Forms. A pesquisa contou com a participação de 80 respondentes, selecionados de forma não probabilística por conveniência, abrangendo pessoas de diferentes perfis e faixas etárias. O questionário foi composto por perguntas relacionadas à frequência de compra de carnes, tempo gasto em compras presenciais, utilização de aplicativos de delivery e interesse em soluções digitais voltadas para o setor de açougues. Os dados coletados foram analisados por meio de estatística descritiva, utilizando frequências absolutas e percentuais para identificação dos principais comportamentos e necessidades dos participantes.

12 RESULTADOS DE PESQUISA COM USUÁRIO

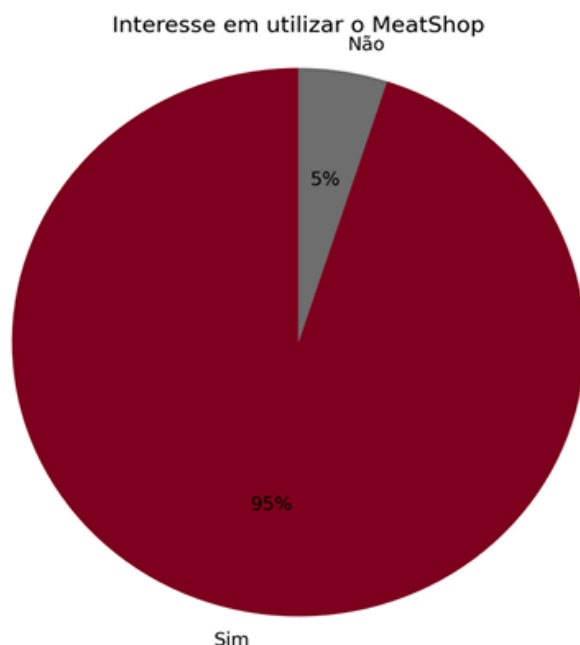
A pesquisa contou com a participação de 80 usuários. Os resultados demonstraram uma forte aceitação da proposta de uma plataforma digital especializada na venda e entrega de produtos de açougue. Os participantes demonstraram elevado interesse na utilização de uma plataforma digital voltada para a compra de carnes e produtos relacionados, indicando que a proposta do MeatShop atende a uma necessidade real do mercado. Em relação ao tempo gasto para realizar compras presenciais em açougues, observou-se que a maior parte dos entrevistados necessita de um

período considerável para concluir o processo de compra. Os resultados reforçam a existência de uma oportunidade para soluções digitais que reduzam deslocamentos, filas e o tempo investido na aquisição dos produtos. Também foi observado que grande parte dos participantes já utiliza aplicativos de compras e delivery em seu cotidiano, demonstrando familiaridade com tecnologias digitais e maior predisposição à adoção de uma plataforma especializada para produtos cárneos.

Perfil da Amostra

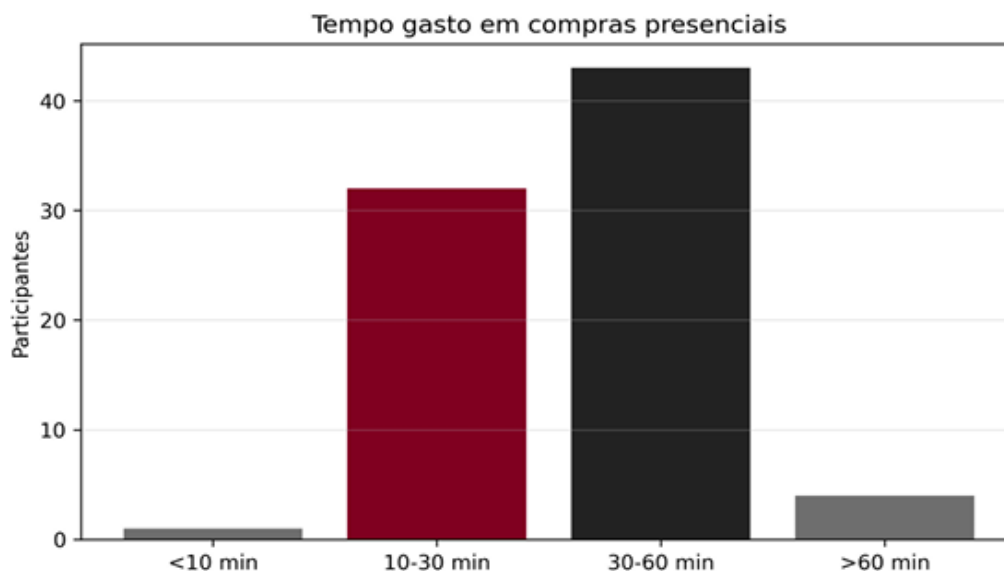
- Total de participantes: 80
- Faixa etária predominante: 18 a 45 anos
- Usuários familiarizados com aplicativos móveis
- Consumidores que realizam compras de alimentos regularmente
- Participantes residentes em áreas urbanas

Figura 1 – Interesse em utilizar o MeatShop



Fonte: Elaborado pelos autores (2026)

Figura 2 – Tempo gasto em compras presenciais



Fonte: Elaborado pelos autores (2026)

Análise dos Resultados

Os gráficos evidenciam que existe forte aceitação da proposta do MeatShop entre os participantes da pesquisa. Além disso, observa-se que a maioria dos entrevistados dedica um tempo significativo às compras presenciais em açougues, o que reforça a necessidade de soluções que proporcionem mais praticidade e conveniência. A combinação desses fatores demonstra que uma plataforma especializada em produtos de açougue possui potencial para melhorar a experiência de compra dos consumidores, reduzindo o tempo gasto no processo e ampliando o acesso aos produtos.

Conclusão

Os dados obtidos demonstram a existência de uma demanda potencial para uma plataforma especializada na comercialização digital de produtos de açougue. O interesse demonstrado pelos participantes e o tempo atualmente investido em compras presenciais validam a proposta do sistema MeatShop. Dessa forma, os resultados da pesquisa contribuíram diretamente para a definição das

funcionalidades implementadas no projeto, garantindo que a solução fosse desenvolvida com foco nas necessidades e expectativas do público-alvo.

13 REGRAS DE NEGÓCIO

ID	Descrição	Requisitos Relacionados
RNE - 001	Cada e-mail poderá estar vinculado a apenas uma conta ativa no sistema.	RF01
Detalhamento: Cada e-mail cadastrado no sistema deve ser único, não sendo permitido o registro de mais de uma conta ativa com o mesmo endereço. Caso um e-mail já esteja registrado, o sistema deverá impedir o novo cadastro e informar ao usuário que a conta já existe.		

ID	Descrição	Requisitos Relacionados
RNE - 002	A senha deverá conter no mínimo 8 caracteres, incluindo letras e números. O armazenamento deverá ocorrer apenas via hash criptográfico seguro.	RF01, RNF07
Detalhamento: As senhas definidas pelos usuários devem seguir critérios mínimos de segurança, contendo no mínimo 8 caracteres com combinação de letras e números. As senhas devem ser armazenadas apenas em formato criptografado (hash), garantindo a proteção das credenciais.		

ID	Descrição	Requisitos Relacionados
RNE - 003	Após 5 tentativas consecutivas de login inválido, a conta deverá ser temporariamente bloqueada por período determinado.	RF01, RNF07
Detalhamento: Para evitar acessos indevidos, o sistema deverá limitar o número de tentativas consecutivas de login. Após cinco tentativas inválidas, a conta deverá ser temporariamente bloqueada, permitindo nova tentativa somente após um período determinado ou verificação adicional.		

ID	Descrição	Requisitos Relacionados
RNE - 004	Apenas produtos com status “ativo” e disponíveis em estoque poderão ser exibidos no catálogo e adicionados ao carrinho.	RF02, RF03
Detalhamento: Apenas produtos com status ativo e disponíveis para venda poderão ser exibidos no catálogo e adicionados ao carrinho de compras. Produtos desativados, removidos ou indisponíveis não devem aparecer para os clientes.		

ID	Descrição	Requisitos Relacionados
RNE - 005	O valor considerado válido para o pedido será sempre o preço vigente no momento da confirmação do checkout.	RF02, RF03, RF09
Detalhamento: O preço considerado válido para a compra será sempre aquele registrado no momento da confirmação do checkout. Caso o valor de um produto seja alterado após sua inclusão no carrinho, o sistema deverá atualizar o preço antes da finalização da compra.		

ID	Descrição	Requisitos Relacionados
RNE - 006	O sistema deverá validar a disponibilidade de estoque antes da finalização do pagamento, impedindo compras acima da quantidade disponível.	RF03, RF04, RNF09
Detalhamento: Antes da confirmação do pagamento, o sistema deverá verificar se há quantidade suficiente em estoque para os produtos selecionados. Caso o estoque seja insuficiente, o sistema deverá impedir a conclusão da compra e informar o usuário.		

ID	Descrição	Requisitos Relacionados
RNE - 007	Um pedido somente poderá ter status confirmado após retorno positivo do provedor de pagamento.	RF04, RNF09
Detalhamento: Um pedido somente poderá ser considerado confirmado após a validação do pagamento pelo provedor responsável. Caso o pagamento seja recusado ou não confirmado, o pedido deverá permanecer com status de pendente ou cancelado.		

ID	Descrição	Requisitos Relacionados
RNE - 008	Todo pedido confirmado deverá ser armazenado de forma persistente, contendo itens, valores, forma de pagamento e data/hora.	RF04, RF09, RNF02
Detalhamento: Todo pedido confirmado deverá ser registrado permanentemente no sistema, incluindo dados como produtos adquiridos, quantidades, valores, forma de pagamento, data e status do pedido.		

ID	Descrição	Requisitos Relacionados
RNE - 009	Os filtros de busca poderão ser aplicados de forma combinada, restringindo progressivamente os resultados.	RF05
Detalhamento: Os filtros de busca disponíveis no catálogo, como categoria, preço, marca e nome do produto, poderão ser aplicados de forma simultânea, permitindo refinar progressivamente os resultados apresentados ao usuário.		

ID	Descrição	Requisitos Relacionados
RNE - 010	A coleta de localização só poderá ocorrer mediante consentimento explícito do usuário.	RF06, RNF07
Detalhamento: A obtenção da localização do dispositivo só poderá ocorrer mediante consentimento explícito do usuário. Caso a permissão seja negada, o sistema deverá oferecer alternativa de inserção manual da localização.		

ID	Descrição	Requisitos Relacionados
RNE - 011	O sistema deverá impedir agendamento fora do horário de funcionamento do estabelecimento.	RF07
Detalhamento: O sistema deve validar o horário selecionado para entrega, garantindo que ele esteja dentro do período de funcionamento e disponibilidade do estabelecimento. Horários indisponíveis não devem ser apresentados para escolha.		

ID	Descrição	Requisitos Relacionados
RNE - 012	Notificações push só poderão ser enviadas para usuários que tenham concedido permissão no dispositivo.	RF08, RNF07
Detalhamento: O envio de notificações push deve ocorrer apenas para usuários que tenham autorizado o recebimento de notificações no dispositivo. As notificações devem estar vinculadas à conta e aos eventos do sistema.		

ID	Descrição	Requisitos Relacionados
RNE - 013	Pedidos finalizados não poderão ser alterados, apenas	RF09, RNF02, RNF11

	consultados ou replicados.	
Detalhamento: Após a finalização de um pedido, os dados da compra não poderão ser alterados. O histórico deverá permanecer apenas para consulta, garantindo integridade e rastreabilidade das transações.		

ID	Descrição	Requisitos Relacionados
RNE - 014	Somente usuários com perfil administrativo poderão acessar o painel de gestão.	RF10, RNF07
Detalhamento: O acesso ao painel administrativo deve ser restrito a usuários com perfil de administrador ou responsável pelo estabelecimento, impedindo acesso indevido a funcionalidades de gestão.		

ID	Descrição	Requisitos Relacionados
RNE - 015	Somente usuários que concluíram um pedido poderão avaliar produtos ou açougues relacionados à compra.	RF11
Detalhamento: Somente usuários que tenham realizado e concluído um pedido poderão registrar avaliações ou comentários sobre produtos ou estabelecimentos, garantindo autenticidade nos feedbacks.		

ID	Descrição	Requisitos Relacionados
RNE - 016	Todas as interações de suporte deverão ser registradas e vinculadas ao usuário responsável.	RF12, RNF11
Detalhamento: Todas as interações realizadas no canal de suporte devem ser registradas no sistema, contendo informações como usuário, data, descrição da solicitação e status do atendimento.		

ID	Descrição	Requisitos Relacionados
RNE - 017	Toda alteração de dados sensíveis deverá gerar registro de log contendo usuário, data, hora e ação executada.	RNF11

Detalhamento: O sistema deve registrar em logs todas as ações críticas, como alterações em produtos, pedidos ou dados de usuários, incluindo identificação do responsável, data e tipo de operação realizada.

ID	Descrição	Requisitos Relacionados
RNE - 018	Relatórios administrativos deverão considerar apenas dados confirmados e persistidos no banco de dados.	RF10, RNF02

Detalhamento: Os relatórios administrativos devem considerar apenas dados confirmados e registrados no sistema, garantindo precisão nas informações apresentadas para análise e tomada de decisão.

ID	Descrição	Requisitos Relacionados
RNE - 019	Em caso de falha de conexão durante o checkout, o pedido deverá permanecer em estado “pendente” até confirmação ou cancelamento automático.	RNF09

Detalhamento: Em caso de falha de conexão durante o processo de compra, o sistema deve preservar os dados já inseridos e permitir que o processo seja retomado ou concluído posteriormente, evitando perda de informações.

ID	Descrição	Requisitos Relacionados
RNE - 020	A aplicação não deverá depender de recursos exclusivos de um único provedor de nuvem para funcionar corretamente.	RNF06, RNF13

Detalhamento: O sistema deve ser desenvolvido de forma a permitir execução em diferentes provedores de nuvem ou ambientes de hospedagem, sem dependência exclusiva de uma infraestrutura específica.

ID	Descrição	Requisitos Relacionados
RNE - 021	As operações principais não deverão exceder 2 segundos em	RNF03, RNF04

	condições normais de uso.	
Detalhamento: As principais operações do sistema devem ser executadas dentro de um tempo máximo aceitável, garantindo respostas rápidas e boa experiência para o usuário.		

ID	Descrição	Requisitos Relacionados
RNE - 022	A indisponibilidade mensal não poderá ultrapassar 0,5% do tempo total.	RNF05
Detalhamento: O sistema deve manter alto nível de disponibilidade, minimizando períodos de indisponibilidade e garantindo acesso contínuo aos usuários.		

14 REQUISITOS FUNCIONAIS

ID	Descrição	Status	Prioridade
RF - 001	O sistema deve permitir o cadastro e login de usuários, garantindo autenticação segura com e-mail/senha ou integração com contas Google e Apple.	Web: Concluído Mobile: Concluído	Alto
Detalhamento: No cadastro com e-mail, o sistema deverá validar os dados informados, garantindo que o e-mail seja único e armazenar a senha utilizando criptografia/hash seguro. No login deverá ser validado as credenciais e, em caso de sucesso, gerar uma sessão/token de autenticação para acesso ao aplicativo e em caso de erro, deverá retornar mensagem de erro sem expor informações sensíveis.			

ID	Descrição	Status	Prioridade
RF - 002	O sistema deve exibir um catálogo atualizado com os produtos disponíveis, mostrando nome, descrição, preço e categoria.	Pronto	Alto
Detalhamento: O sistema deve disponibilizar um catálogo de produtos atualizado, exibindo as informações de cada item: nome, descrição, preço e categoria. Os dados apresentados devem refletir as informações cadastradas no sistema, garantindo consistência e atualização em tempo real			

ou conforme a política de sincronização definida.

ID	Descrição	Status	Prioridade
RF - 003	O sistema deve permitir adicionar, remover e atualizar produtos no carrinho, exibindo o total e quantidade de itens.	Pronto	Alto
Detalhamento: O sistema deve permitir que o usuário adicione, remova e atualize a quantidade de produtos no carrinho de compras antes da finalização do pedido. Sempre que houver alteração, o sistema deverá atualizar automaticamente o valor total da compra e a quantidade total de itens, garantindo consistência nos cálculos.			

ID	Descrição	Status	Prioridade
RF - 004	O sistema deve oferecer opções de pagamento via cartão ou PIX exibindo confirmação após a transação.	Backend: Pronto Mobile: Ainda não realizado	Alto
Detalhamento: O sistema deve permitir que o usuário finalize a compra por meio de um processo de checkout seguro, oferecendo opções de pagamento via cartão de crédito/débito ou PIX. Durante o checkout, o sistema apresentará o resumo do pedido, incluindo produtos, quantidades e valor total, antes da confirmação do pagamento, após o pagamento uma mensagem clara de confirmação ou falha será, registrando o status da operação no pedido.			

ID	Descrição	Status	Prioridade
RF - 005	O sistema deve permitir busca e filtros por nome, categoria, preço e marca, retornando resultados em tempo real.	Pronto	Alto
Detalhamento: O sistema deve permitir que o usuário realize busca por nome de produto e aplique filtros por categoria, faixa de preço e marca, facilitando a localização de itens específicos no catálogo onde os resultados devem ser atualizados em tempo real ou dinamicamente após a aplicação dos filtros, exibindo apenas produtos que atendam aos critérios selecionados além disso a busca deve considerar correspondência parcial de texto (ex.: parte do nome) e respeitar apenas			

produtos ativos/disponíveis.

ID	Descrição	Status	Prioridade
RF - 006	Sistema que puxa a localização do aparelho para saber os açougues mais próximos.	Ainda não iniciado (Será realizado no Mobile)	Alto

Detalhamento: O sistema deve utilizar a localização do dispositivo (mediante autorização do usuário) para identificar e exibir os açougues mais próximos disponíveis na plataforma onde a aplicação vai obter as coordenadas geográficas do aparelho e calcular a proximidade com base nos estabelecimentos cadastrados, apresentando os resultados ordenados por distância e caso o usuário negue a permissão de localização, o sistema deverá permitir a inserção manual da cidade ou região como alternativa.

ID	Descrição	Status	Prioridade
RF - 007	O sistema deve permitir ao cliente escolher data e horário para entrega dos pedidos conforme disponibilidade.	Em andamento	Alto

Detalhamento: O sistema deve permitir que o cliente selecione data e horário disponíveis para entrega no momento da finalização do pedido. As opções exibidas devem respeitar a disponibilidade do estabelecimento, considerando dias de funcionamento, horários permitidos e possíveis limitações de capacidade por período, além disso o sistema deverá validar a escolha antes da confirmação do pedido, impedindo a seleção de horários indisponíveis ou já esgotados.

ID	Descrição	Status	Prioridade
RF - 008	O sistema deve enviar notificações push sobre promoções, atualizações e status de pedidos.	Em andamento	Médio

Detalhamento: O sistema deve enviar notificações push aos usuários para informar sobre promoções, atualizações relevantes e alterações no status dos pedidos. As notificações devem ser enviadas mediante autorização do usuário e associadas à sua conta/dispositivo autenticado.

ID	Descrição	Status	Prioridade
----	-----------	--------	------------

RF - 009	O sistema deve exibir o histórico de pedidos realizados, com detalhes e opção de repetir compras.	Ainda não iniciado	Médio
----------	---	--------------------	-------

Detalhamento: O sistema deve permitir que o usuário visualize o histórico de pedidos já realizados, exibindo informações como data da compra, status, valor total e itens adquiridos. Cada pedido deverá apresentar detalhamento completo, incluindo produtos, quantidades, preços e forma de pagamento utilizada, o sistema deve disponibilizar a opção de repetir a compra, permitindo adicionar automaticamente os mesmos itens ao carrinho, respeitando disponibilidade e possíveis alterações de preço.

ID	Descrição	Status	Prioridade
RF - 010	O sistema deve disponibilizar um painel para o administrador gerenciar produtos, pedidos e promoções, além de visualizar relatórios.	Pronto	Alto

Detalhamento: O sistema deve disponibilizar um painel administrativo restrito ao açougue/administrador, acessível mediante autenticação com perfil autorizado. O painel deve permitir o gerenciamento de produtos, incluindo cadastro, edição, ativação/desativação e atualização de preços e categorias, também deverá possibilitar o acompanhamento e atualização do status dos pedidos, bem como a criação e gestão de promoções.

ID	Descrição	Status	Prioridade
RF - 011	O sistema deve permitir que clientes avaliem produtos e açougues com notas e comentários.	Ainda não iniciado	Baixo

Detalhamento: O sistema deve permitir que clientes autenticados realizem avaliações de produtos e dos açougues, atribuindo notas e comentários após a conclusão de uma compra. As avaliações devem ficar vinculadas ao pedido realizado, garantindo que apenas clientes que efetivamente compraram possam avaliar.

ID	Descrição	Status	Prioridade
RF - 012	O sistema deve oferecer um canal de contato (chat ou formulário) para dúvidas e suporte.	Ainda não iniciado	Baixo

Detalhamento: O sistema deve disponibilizar um canal de atendimento ao cliente, por meio de chat online ou formulário de contato, para esclarecimento de dúvidas, registro de solicitações e suporte pós-venda. O canal deve permitir que o usuário envie sua mensagem informando assunto, descrição do problema e dados de contato, quando necessário.

15 REQUISITOS NÃO FUNCIONAIS

ID	Descrição	Status	Prioridade
RNF - 001	O aplicativo deve funcionar corretamente em dispositivos Android, iOS e navegadores web.	Em andamento	Alta
Detalhamento: O sistema deve funcionar corretamente em dispositivos Android, iOS e navegadores web, garantindo experiência consistente entre as plataformas. A aplicação deve ser compatível com versões recentes dos sistemas operacionais e principais navegadores de mercado, assegurando estabilidade, layout responsivo e pleno funcionamento das funcionalidades.			

ID	Descrição	Status	Prioridade
RNF - 002	Todos os dados devem ser armazenados de forma segura em servidores e bancos de dados persistentes.	Banco: Pronto Mobile: Em andamento	Alta
Detalhamento: O armazenamento deve assegurar que dados de usuários, pedidos pagamentos e produtos não serão perdidos em caso de falhas ou reinicializações do sistema.			

ID	Descrição	Status	Prioridade
RNF - 003	O sistema deve manter tempo de resposta estável mesmo sob alta carga de usuários simultâneos.	Pronto	Alta

Detalhamento: As operações principais, como busca de produtos, atualização do carrinho e finalização de pedidos, devem ocorrer dentro de limites aceitáveis de tempo, evitando atrasos perceptíveis ao usuário. A arquitetura deve permitir escalabilidade, suportando aumento de acessos sem degradação significativa da performance, por meio de otimizações, balanceamento de carga ou expansão de infraestrutura quando necessário.

ID	Descrição	Status	Prioridade
RNF - 004	As páginas e ações principais devem carregar em até 2 segundos em rede 4G.	Em andamento	Média

Detalhamento: Operações críticas, como carregamento do catálogo, atualização do carrinho e abertura do checkout, devem ser otimizadas para garantir experiência fluida e sem atrasos perceptíveis.

ID	Descrição	Status	Prioridade
RNF - 005	O sistema deve permanecer ativo e acessível 99,5% do tempo mensal.	Pronto	Alta

Detalhamento: Eventuais períodos de indisponibilidade devem ser planejados, preferencialmente fora de horários de pico, e comunicados previamente quando se tratar de manutenção programada. A infraestrutura deve contar com mecanismos de monitoramento, redundância e recuperação de falhas, reduzindo riscos de interrupções prolongadas.

ID	Descrição	Status	Prioridade
RNF - 006	A arquitetura deve permitir expansão de recursos (usuários, pedidos, produtos) sem perda de desempenho.	Em andamento	Alta

Detalhamento: O sistema deve ser projetado de forma modular e preparada para crescimento, possibilitando ampliação de capacidade por meio de ajustes na infraestrutura, balanceamento de carga ou distribuição de serviços. A escalabilidade deve garantir que o aumento de demanda não gere degradação significativa do tempo de resposta ou indisponibilidade do sistema.

ID	Descrição	Status	Prioridade
RNF - 007	O sistema deve proteger dados pessoais e estar em conformidade com a Lei Geral de Proteção de Dados (LGPD).	Ainda não iniciado	Alta

Detalhamento: As informações sensíveis devem ser armazenadas de forma segura, com uso de criptografia, controle de acesso e autenticação adequada, o sistema deve estar em conformidade com a Lei Geral de Proteção de Dados (LGPD), assegurando a transparência na coleta e uso dos dados, possibilidade de atualização ou exclusão mediante solicitação do titular e registro adequado do consentimento.

ID	Descrição	Status	Prioridade
RNF - 008	A interface deve ser simples, intuitiva e permitir concluir uma compra em até 5 cliques.	Em andamento	Alta

Detalhamento: O fluxo de compra deve ser otimizado para que o cliente consiga concluir um pedido em até 5 cliques, desde a seleção do produto até a confirmação do pagamento, sempre que possível. Os elementos visuais devem ser organizados de forma clara, com botões identificáveis, mensagens objetivas e navegação consistente entre telas, garantindo boa experiência para diferentes perfis de usuários.

ID	Descrição	Status	Prioridade
RNF - 009	O sistema deve preservar os pedidos mesmo que ocorram falhas de conexão ou interrupções temporárias.	Em andamento	Média

Detalhamento: Em caso de instabilidade de rede durante o processo de compra, o sistema deve manter as informações já registradas, evitando perda de dados e

possibilitando a retomada da operação. A arquitetura deve prever mecanismos como salvamento automático, controle transacional recuperação de estado, assegurando consistência das informações e evitando duplicidade ou inconsistência nos pedidos.

ID	Descrição	Status	Prioridade
RNF- 010	O código deve ser modular, bem documentado e seguir padrões de boas práticas.	Documentação Contínua	Média

Detalhamento: O código-fonte deve ser bem documentado, organizado e seguir padrões de boas práticas de desenvolvimento, como convenções de nomenclatura, separação de responsabilidades e versionamento adequado, bem como a estrutura do projeto deve permitir futuras melhorias, integrações e refatorações com baixo risco, garantindo sustentabilidade técnica a longo prazo.

ID	Descrição	Status	Prioridade
RNF - 011	O sistema deve registrar todas as ações críticas de usuários e administradores.	Em andamento	Alta

Detalhamento: Devem ser registrados eventos como login, alterações de dados, criação e cancelamento de pedidos, mudanças de status, modificações em produtos e ações administrativas relevantes. Os registros de log devem conter informações como data, horário, usuário responsável e descrição da ação, sendo armazenados de forma segura e protegidos contra alterações indevidas.

ID	Descrição	Status	Prioridade
RNF - 012	O sistema deve permitir o acompanhamento em tempo real de métricas, erros e desempenho.	Pronto	Média

Detalhamento: Devem ser monitorados indicadores como tempo de resposta, taxa de erros, disponibilidade, uso de recursos e volume de requisições. A solução deve incluir mecanismos de alerta automático em caso de comportamento anormal, possibilitando atuação preventiva da equipe técnica.

ID	Descrição	Status	Prioridade
RNF - 013	O sistema deve ser compatível com diferentes ambientes e provedores de nuvem.	Ainda não iniciado	Média
Detalhamento: A aplicação deve suportar execução em ambientes de desenvolvimento, homologação e produção, mantendo comportamento consistente entre eles. A arquitetura e tecnologias permitirão a migração entre provedores de nuvem ou servidores distintos com impacto mínimo, assegurando flexibilidade e independência tecnológica.			

16 RASTREABILIDADE DE REQUISITOS

A rastreabilidade de requisitos foi elaborada para demonstrar a ligação entre os problemas identificados no contexto do MeatShop e as funcionalidades propostas ou implementadas no sistema. Essa tabela permite acompanhar a trilha lógica entre dor de mercado, requisito, regra de negócio, tela, evidência de teste e status de implementação.

ID	Problema de origem	Requisito(s)	Regra(s) de negócio	Tela/Funcionalidade implementada	Teste/Evidência	Status
RT-01	Atendimento presencial demorado e dependente da disponibilidade do açougueiro.	RF-002, RF-003	RNE-004, RNE-006	Catálogo de produtos, detalhes do produto e carrinho.	Validação de cadastro de produtos, regras de estoque e fluxo de carrinho.	Implementado / em validação contínua
RT-02	Necessidade de deslocamento físico até o açougue para consultar e comprar carnes.	RF-002, RF-004, RF-007	RNE-005, RNE-007, RNE-011	Listagem de açougues, catálogo, checkout, seleção de entrega e pagamento.	Evidências das telas mobile e integração de pagamento no backend.	Parcialmente implementado
RT-03	Falta de controle e registro formal dos pedidos recebidos pelo açougue.	RF-009, RF-010	RNE-008, RNE-013, RNE-018	Painel web de pedidos, histórico e relatórios administrativos.	Registros persistidos no banco e telas de acompanhamento administrativo.	Implementado parcialmente
RT-04	Risco de venda de produtos sem estoque suficiente.	RF-003, RF-004	RNE-004, RNE-006	Carrinho, checkout e validação de disponibilidade.	Testes de produto com preço/estoque inválidos e validações de regra.	Implementado
RT-05	Cliente sem visibilidade clara	RF-008, RF-009	RNE-012, RNE-013	Acompanhamento de pedido, notificações e	Evidências do fluxo mobile de	Em andamento /

	sobre andamento do pedido e entrega.			rastreamento em tempo real.	acompanhamento e mapa interativo.	funcionalidade avançada
RT-06	Necessidade de pagamento online seguro para concluir pedidos.	RF-004	RNE-007, RNE-008, RNE-019	Checkout, pagamento via Mercado Pago, confirmação ou falha de transação.	Integração Mercado Pago no backend desde a versão 1.1.0.	Backend implementado / mobile a finalizar
RT-07	Risco de acesso indevido a contas e dados de usuários.	RF-001, RNF-007	RNE-001, RNE-002, RNE-003, RNE-014	Login, cadastro, autenticação JWT, senha com hash e bloqueio por tentativas.	Testes de autenticação, validação de senha e fluxo de bloqueio temporário.	Implementado parcialmente
RT-08	Exposição de dados pessoais como CPF, endereço, localização e histórico de pedidos.	RNF-002, RNF-007, RNF-011	RNE-010, RNE-012, RNE-017	Controle de permissões, logs de ações críticas, consentimento de localização e notificações.	Evidência documental de LGPD e segurança; logs e regras de acesso.	A reforçar na documentação
RT-09	Dificuldade do açougue em organizar produtos, promoções e estoque.	RF-010	RNE-014, RNE-018	Painel administrativo web, gestão de produtos, estoque, promoções e financeiro.	Telas web implementadas e histórico de versões do sistema.	Implementado
RT-10	Ausência de apoio ao cliente sobre cortes, receitas e preparo de carnes.	RF-012	RNE-016	Chatbot especializado em carnes, cortes, receitas e técnicas de preparo.	API de chatbot MeatShop integrada ao mobile em estágio inicial.	Em amadurecimento
RT-11	Atendimento presencial demorado e dependente da disponibilidade do açougueiro.	RF-006, RF-007	RNE-010, RNE-011	Localização do aparelho, seleção de endereço e agendamento de entrega.	Permissão de localização e alternativa de inserção manual previstas.	Planejado / em andamento
RT-12	Necessidade de deslocamento físico até o açougue para consultar e comprar carnes.	RNF-003, RNF-004, RNF-005, RNF-012	RNE-021, RNE-022	Métricas com Prometheus/Grafana, testes de carga e observabilidade.	Resultados de testes K6, tempo médio, p95/p99, taxa de erro e throughput.	Implementado / documentação a complementar

17 FUNDAMENTAÇÃO TEÓRICA

17.1 Transformação digital no varejo e oportunidade de negócio

A transformação digital no varejo representa o uso de tecnologias digitais para reorganizar processos de atendimento, venda, controle operacional e relacionamento com clientes. Em pequenos e médios estabelecimentos, como açougues, essa transformação permite substituir práticas predominantemente manuais ou informais por fluxos mais organizados, rastreáveis e integrados. No contexto do MeatShop, essa lógica se aplica à digitalização do atendimento, do catálogo de produtos, do controle de estoque, dos pedidos, dos pagamentos e da comunicação entre cliente, açougue e entregador.

O problema identificado no projeto demonstra que muitos consumidores ainda dependem do deslocamento físico até o açougue, enfrentando filas, indisponibilidade de produtos e possíveis erros de comunicação durante o atendimento. Ao propor uma plataforma digital especializada em açougues, o MeatShop transforma a dor cotidiana em uma oportunidade de negócio: ampliar o alcance dos estabelecimentos, reduzir o tempo gasto pelo cliente e agregar valor por meio de funcionalidades como pedido online, agendamento, pagamento digital, acompanhamento da entrega e apoio inteligente por chatbot especializado em carnes.

17.2 E-commerce, delivery especializado e valor agregado ao cliente

O comércio eletrônico pode ser compreendido como a realização de transações comerciais mediadas por tecnologias digitais, envolvendo apresentação de produtos, escolha, compra, pagamento e acompanhamento do pedido. Diferentemente de aplicativos generalistas de delivery, que concentram grande parte da operação em refeições prontas, o MeatShop se posiciona em um nicho específico: a compra de carnes e itens de açougue para preparo doméstico, churrascos, refeições familiares e abastecimento de pequenos negócios.

Esse posicionamento fortalece o valor agregado da solução, pois o sistema não se limita a entregar produtos. Ele organiza uma experiência completa, permitindo que o cliente visualize o catálogo, consulte produtos disponíveis, monte o carrinho, realize o pagamento, acompanhe o pedido e receba orientações sobre cortes, receitas e preparo. Para o açougue, a solução agrega controle administrativo, gestão de pedidos, estoque, promoções e relatórios, contribuindo para uma operação mais profissional e orientada a dados.

17.3 Experiência do usuário, usabilidade e acessibilidade

A experiência do usuário é um elemento central para a aceitação de sistemas digitais, pois envolve a forma como o usuário percebe, compreende e utiliza uma aplicação para alcançar seus objetivos. No MeatShop, o público-alvo inclui adultos com rotina corrida, pessoas que vivem sozinhas ou em casal e usuários já familiarizados com aplicativos de compra e delivery. Portanto, a interface precisa ser

objetiva, clara e eficiente, reduzindo etapas desnecessárias e facilitando a conclusão da compra.

A usabilidade do sistema se relaciona com a facilidade de navegação, clareza dos botões, organização das telas, mensagens de erro compreensíveis e consistência visual entre os fluxos. No aplicativo mobile, isso se reflete em telas como catálogo, detalhes do produto, carrinho, seleção de endereço, pagamento, acompanhamento de entrega e chat. No sistema web, a usabilidade aparece no painel administrativo, no gerenciamento de produtos, pedidos, estoque e relatórios financeiros.

Além da usabilidade, a acessibilidade deve ser considerada para que diferentes perfis de usuários consigam utilizar a solução. As diretrizes WCAG 2.2 organizam a acessibilidade em quatro princípios: conteúdo perceptível, operável, compreensível e robusto (W3C, 2024). Aplicando esses princípios ao MeatShop, é importante manter contraste adequado entre textos e fundos, botões identificáveis, mensagens claras, campos com rótulos compreensíveis, navegação previsível e suporte a diferentes tamanhos de tela.

17.4 Engenharia de requisitos e rastreabilidade

A engenharia de requisitos tem como finalidade identificar, documentar, organizar e validar as necessidades que um sistema deve atender. No MeatShop, as dores levantadas no problema de pesquisa, como atendimento demorado, deslocamento até o açougue, falta de controle de pedidos e necessidade de rastreamento, foram convertidas em requisitos funcionais, requisitos não funcionais e regras de negócio.

A rastreabilidade de requisitos é importante porque demonstra a relação entre a origem do problema, a funcionalidade proposta, a regra aplicada, a tela desenvolvida, a evidência de teste e o status de implementação. De acordo com a ISO/IEC/IEEE 29148:2018, os processos de requisitos devem apoiar a definição, documentação e acompanhamento das necessidades do sistema ao longo do ciclo de vida do produto (ISO/IEC/IEEE, 2018). Dessa forma, a rastreabilidade reduz ambiguidades e facilita a avaliação da banca sobre a coerência entre problema, solução e implementação.

No projeto MeatShop, essa rastreabilidade contribui para mostrar que as funcionalidades não foram criadas de forma isolada. O catálogo de produtos responde à dificuldade de consultar disponibilidade; o carrinho e o checkout respondem à necessidade de compra online; o acompanhamento de entrega responde à demanda por visibilidade do pedido; o painel administrativo responde à necessidade de controle do açougue; e os mecanismos de segurança respondem à necessidade de proteção de dados pessoais e transacionais.

17.5 Arquitetura de software em camadas

A arquitetura de software define a organização estrutural de um sistema, suas partes principais e as relações entre elas. Em sistemas que tendem a crescer, a separação de responsabilidades facilita a manutenção, testes, evolução e substituição de componentes. O MeatShop foi estruturado com uma arquitetura em camadas, separando apresentação, regras de negócio, persistência de dados e observabilidade.

Na camada de apresentação, estão o aplicativo mobile desenvolvido em Flutter e o sistema web desenvolvido em Next.js. Essas interfaces são responsáveis por exibir informações, capturar interações e enviar requisições ao backend. A camada de aplicação, desenvolvida em NestJS, centraliza regras de negócio, autenticação, validação, pedidos, produtos, integração com pagamento e comunicação com o banco de dados. A camada de persistência utiliza PostgreSQL e Firebase conforme as necessidades de dados estruturados, sincronização e recursos em tempo real. A camada de observabilidade, com Prometheus e Grafana, contribui para o acompanhamento de métricas, desempenho e disponibilidade.

Essa divisão está alinhada à necessidade de manter o projeto compreensível e evolutivo. A separação entre frontend, backend e banco evita acesso direto e descontrolado aos dados, melhora a segurança e permite que novas funcionalidades, como notificações, chatbot, rastreamento e relatórios, sejam integradas de forma progressiva.

17.6 Segurança da informação e proteção de dados

Como o MeatShop manipula dados pessoais e operacionais, a segurança da informação deve ser tratada como requisito essencial do projeto. Entre os dados envolvidos estão nome, e-mail, CPF, telefone, endereço, localização, histórico de pedidos, dados de entrega e registros relacionados ao pagamento. A Lei Geral de Proteção de Dados Pessoais dispõe sobre o tratamento de dados pessoais, inclusive em meios digitais, com o objetivo de proteger direitos fundamentais de liberdade, privacidade e livre desenvolvimento da personalidade (BRASIL, 2018).

No projeto, as medidas técnicas já descritas incluem autenticação com JWT, controle de sessão, uso de refresh token, hash de senhas com bcrypt, controle de permissões por perfil, regras de acesso, logs de ações críticas e comunicação segura com serviços externos. A segurança também deve considerar princípios como minimização de dados, controle de acesso, transparência no uso das informações, proteção contra exposição indevida e possibilidade de correção ou exclusão de dados mediante solicitação do titular.

17.7 Síntese da fundamentação aplicada ao MeatShop

A fundamentação teórica demonstra que o MeatShop não é apenas uma aplicação de delivery, mas uma solução digital especializada para um problema real do varejo de carnes. A proposta combina transformação digital, e-commerce, experiência do usuário, arquitetura em camadas, engenharia de requisitos, segurança da informação e proteção de dados pessoais.

Dessa forma, o projeto se justifica pela capacidade de gerar valor para dois públicos principais. Para o cliente, oferece praticidade, economia de tempo, compra online, acompanhamento do pedido e apoio por inteligência artificial. Para o açougue, oferece organização operacional, controle de estoque, gerenciamento de pedidos, relatórios, maior alcance comercial e redução de erros no atendimento.

PARTE 04 – DESIGN, MODELAGEM, DADOS E SOLUÇÃO TECNOLÓGICA

18 DESIGN DA SOLUÇÃO E EXPERIÊNCIA DO USUÁRIO

18.1 Fluxo de Navegação

Mobile

O usuário entra na aplicação, vai para a tela de login onde caso ele não possua uma conta ele será direcionado para cadastro de usuário, após isso ou caso já esteja cadastrado ele realiza o login no sistema onde ele vai para a home onde ele consegue ver o catálogo de produtos e quais se encontram em promoção, então ele filtra e escolhe os produtos que deseja e vai para o carrinho, onde realiza o pagamento e encerra a compra, dessa forma, podendo acompanhar o pedido e saber onde ele se encontra, além de este caminho o usuário também poderá deixar um pedido agendado antecipadamente e quando chega a hora o pedido é enviado ao açougue realizar a fluxo padrão.

Fluxo de navegação web

O dono do açougue realiza o login na conta do estabelecimento então checa os pedidos disponíveis quais foram atendidos e quais estão em espera, ele pode visualizar os estoques de produtos e atualizá-los, pode ver relatórios das despesas e lucros do mês bem como inserir promoções nos produtos a venda.

18.2 Protótipos do Sistema

O protótipo das telas do mobile desenvolvido pode ser acessado utilizando o endereço: <https://shre.ink/3Haa>

19 GESTÃO DO PROJETO DE MODELAGEM DO SISTEMA

19.1 Organização do Projeto (Abordagem Ágil)

Utilizamos do método Scrumban que mistura a estrutura ágil e focada em fluxo contínuo de entregas com a visualização em kanban onde dividimos o que precisa ser feito o que está sendo feito e o que já foi feito com o auxílio da ferramenta trello para gestão de projetos e tarefas além de também realizarmos dailys

semanais para um resumo e organização do que foi produzido essa semana e o que será para a próxima.

19.2 Modelagem e Diagramas do Sistema

Para ver o como foi diagramado o fluxo do Sistema basta acessar este endereço: <https://shre.ink/3HcG>

O usuário primeiro se cadastra ou realiza o login. Depois disso, o acesso muda conforme o tipo de perfil: pode ser, cliente, entregador, ou até os dois ao mesmo tempo. O cliente escolhe produtos, monta o pedido, define pagamento e acompanha o andamento até a entrega ou retirada. A unidade recebe esse pedido, confirma, prepara e atualiza o estoque. Se precisar de entrega, o sistema procura um entregador disponível para concluir a corrida. Já o entregador precisa estar aprovado e ativo para receber pedidos. Além disso, usuários vinculados a uma unidade podem atuar internamente como admin ou membro, ajudando na gestão de produtos, estoque e pedidos. Durante todo o processo, o sistema também envia notificações e pode abrir chat entre cliente, unidade e suporte.

20 DADOS E MODELAGEM DA INFORMAÇÃO

20.1 Modelo de Dados

Aqui pode se ver o diagrama DER do projeto acessando o endereço: <https://shre.ink/3Hcf>

Esse diagrama representa a base estrutural do sistema, mostrando as principais entidades e como elas se conectam. A entidade central é User, que armazena os dados do usuário e define seu papel no sistema, podendo atuar como cliente, entregador, ou membro de uma unidade. Além disso, um usuário pode ter vários endereços e também se vincular a unidades por meio da entidade UserUnit, que define sua função local, como administrador ou membro operacional. A entidade Unit representa as unidades ou lojas do sistema. Cada unidade possui seu próprio

catálogo, organizado por Category e Product, além do controle de Stock, que registra a quantidade disponível de cada produto. Essa estrutura permite que cada unidade administre seus itens, categorias e estoque de forma independente, mantendo a organização do catálogo e do funcionamento interno da operação. Já o fluxo de compra é representado principalmente por Order, OrderItem e Payment, que registram o pedido, os itens comprados e o pagamento realizado. Quando há entrega, entra também a entidade DeliveryPerson, responsável pelo entregador, e a DeliveryReview, usada para avaliação após a entrega. Complementando a experiência do sistema, as entidades Notification e Chat cuidam da comunicação, permitindo o envio de avisos e a troca de mensagens relacionadas ao pedido.

20.2 Estrutura e Manipulação de Dados

Os dados do sistema MeatShop são armazenados de forma estruturada em um banco de dados relacional, utilizando o PostgreSQL como tecnologia principal de persistência. Essa escolha permite organizar as informações em entidades relacionadas entre si, garantindo consistência, integridade e rastreabilidade dos dados manipulados pela aplicação. A modelagem foi construída para refletir os principais fluxos do projeto, como cadastro de usuários, gerenciamento de unidades, catálogo de produtos, controle de estoque, criação de pedidos, pagamentos, entregas, notificações e comunicação via chat.

A organização dos dados segue a lógica de separação por domínio de negócio. As informações de usuários ficam centralizadas na entidade User, enquanto seus vínculos com unidades são controlados por UserUnit, permitindo diferenciar permissões e papéis locais dentro do sistema. Os dados operacionais dos açougues são organizados por Unit, Category, Product e Stock, possibilitando que cada unidade mantenha seu próprio catálogo e seu próprio controle de disponibilidade. Já o fluxo de compra é estruturado a partir das entidades Order, OrderItem e Payment, responsáveis por registrar o pedido, os itens comprados, valores, status e confirmação do pagamento. Para o processo de entrega, a entidade DeliveryPerson armazena os dados do entregador, enquanto DeliveryReview, Notification e Chat complementam a experiência do usuário com

avaliação, comunicação e acompanhamento do pedido.

A manipulação desses dados ocorre por meio do backend, que atua como camada intermediária entre as interfaces do sistema e o banco de dados. Toda inserção, consulta, atualização ou exclusão passa pelas regras de negócio implementadas na API, evitando acesso direto e descontrolado às informações persistidas. Isso permite validar dados antes do salvamento, como unicidade de e-mail, disponibilidade de estoque, confirmação de pagamento, controle de status do pedido e permissões de acesso. Dessa forma, a estrutura de dados não funciona apenas como armazenamento, mas também como suporte direto à lógica do projeto e à execução segura dos fluxos principais da aplicação.

Essa organização está diretamente relacionada aos requisitos funcionais e não funcionais definidos no projeto. O catálogo de produtos, carrinho, checkout, histórico de compras e painel administrativo dependem da correta associação entre produtos, pedidos, pagamentos e usuários. Da mesma forma, requisitos como persistência dos dados, confiabilidade, auditabilidade e segurança são atendidos por meio do armazenamento estruturado, do uso de chaves primárias e estrangeiras, do controle de integridade entre tabelas e do registro consistente das operações realizadas. Assim, a estrutura e a manipulação de dados do MeatShop foram planejadas para sustentar o funcionamento do sistema de forma organizada, segura e alinhada às necessidades do negócio.

21 SOLUÇÃO TECNOLÓGICA E EXPERIMENTAÇÃO TÉCNICA INICIAL

21.1 Escolhas Tecnológicas

Frontend Web – Next.js O painel administrativo será construído em Next.js por oferecer flexibilidade e alta performance, enquanto reduz erros e facilita a manutenção do código em um projeto que tende a crescer.

Backend – Em Camadas (Node.js/NestJS) que centraliza o backend em um único sistema, mas mantém separação por módulos de domínio (auth, catálogo, pedidos, estoque, notificações, analytics). Isso facilita a organização e prepara o sistema

para futura evolução

Banco de Dados Relacional – PostgreSQL O PostgreSQL foi definido como banco transacional (OLTP), pela robustez, escalabilidade e suporte à integridade de dados. É amplamente utilizado em sistemas de e-commerce e tem excelente integração com ORMs e ferramentas de análise.

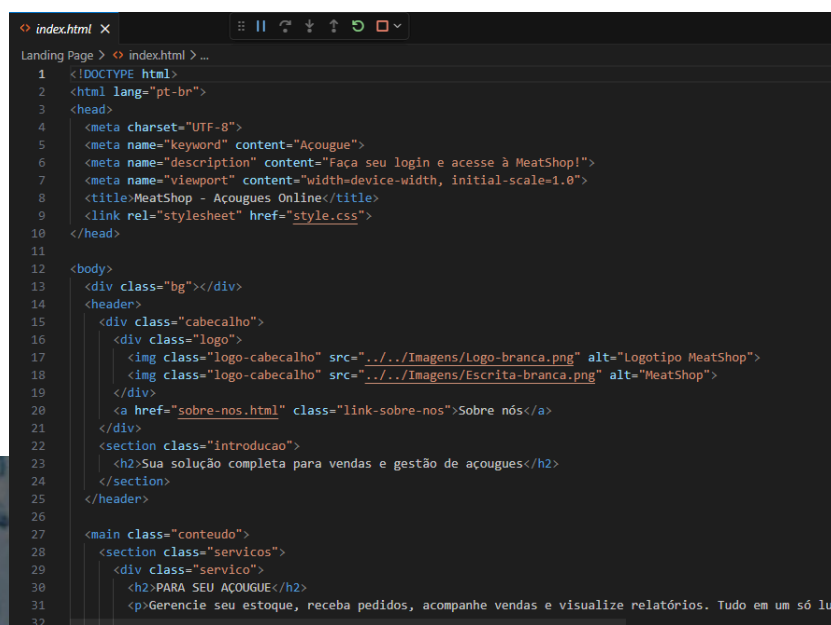
DevOps – GitHub Actions e CI/CD O GitHub Actions foi definido para automação de CI/CD, permitindo pipelines com lint, testes e builds automatizados a cada push. Essa escolha garante rastreabilidade e agilidade no desenvolvimento contínuo.

Frontend Mobile – Flutter foi definido para o aplicativo do cliente por permitir desenvolver aplicações nativas para Android e iOS a partir de um único código-fonte. Essa escolha reduz o esforço de desenvolvimento, garante boa performance e conta com suporte de uma comunidade ativa.

21.2 Experimentação Técnica e Primeiros Testes

Iniciaram-se as experimentações utilizando CSS básico com JavaScript, criando uma tela de landing page e login do açougue. Os testes foram bem sucedidos, as telas ficaram bonitas e funcionais, porém posteriormente constatou-se que a linguagem escolhida seria mais trabalhosa de executar e decidimos trocar. Entretanto, de maneira geral, as primeiras experimentações de código no Front-end surpreenderam positivamente, trazendo uma interface e um código limpos.

Figura 3 - Implementação de uma landing page do açougue



```
1 <!DOCTYPE html>
2 <html lang="pt-br">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="keyword" content="Açougue">
6   <meta name="description" content="Faça seu login e acesse à MeatShop!">
7   <meta name="viewport" content="width=device-width, initial-scale=1.0">
8   <title>MeatShop - Açougues Online</title>
9   <link rel="stylesheet" href="style.css">
10 </head>
11
12 <body>
13   <div class="bg"></div>
14   <header>
15     <div class="cabecalho">
16       <div class="logo">
17         
18         
19       </div>
20       <a href="sobre-nos.html" class="link-sobre-nos">Sobre nós</a>
21     </div>
22     <section class="introducao">
23       <h2>Sua solução completa para vendas e gestão de açougues</h2>
24     </section>
25   </header>
26
27   <main class="conteudo">
28     <section class="servicos">
29       <div class="servico">
30         <h2>PARA SEU AÇOUGUE</h2>
31         <p>Gerencie seu estoque, receba pedidos, acompanhe vendas e visualize relatórios. Tudo em um só lugar</p>
32       </div>
33     </section>
34   </main>
35 </body>
36 </html>
```

Fonte: Elaborado pelos autores (2026).

Figura 4 - Implementação de uma tela de login do açougue

```
33 <section class="acessos">
34   <a href="../../Cadastro/index-acougue.html" class="botao">Cadastre-se</a>
35 </section>
36
37 <p class="login-text">Já tem uma conta? <a href="../../Açougue/Login/index.html" class="link-login">Faça login</a></p>
38 </div>
39
40 <div class="servico">
41   <h2>PARA CLIENTES</h2>
42   <p>Encomende carnes com facilidade, veja promoções, avalie açougues e agende retiradas. Rápido, prático e seguro.</p>
43
44   <section class="acessos">
45     <a href="../../Cadastro/index-cliente.html" class="botao">Cadastre-se</a>
46   </section>
47
48   <p class="login-text">Já tem uma conta? <a href="../../Cliente/Login/index.html" class="link-login">Faça login</a></p>
49 </div>
50 </section>
51
52 <section class="contato">
53   <h2>Fale Conosco</h2>
54   <form>
55     <input type="text" placeholder="Seu nome" required>
56     <input type="email" placeholder="Seu e-mail" required>
57     <textarea rows="4" placeholder="Sua mensagem" required></textarea>
58     <button type="submit">Enviar</button>
59   </form>
60 </section>
61 </main>
62
63 <footer>
64   <p>© 2025 MeatShop. Todos os direitos reservados.</p>
65 </footer>
66
67 <script src="js/script.js"></script>
68 </body>
69 </html>
70
```

Fonte: Elaborado pelos autores (2026).

Backend experimentação

Esse código foi feito para fazer uma simulação de um registro simples salvando os dados em um banco de dados com a segurança de hash de senhas do bcrypt

register.dto.ts:

Ele define quais dados a rota do cadastro tem que receber e as regras de validação que esses dados devem seguir

auth.module.ts:

Esse módulo agrupa tudo que pertence à autenticação como o UsersModule,

AuthController, AuthService

users.entity.ts:

Essa entidade cria a classe User e define seus atributos

auth.controller.ts:

É o controller de autenticação, a parte que recebe a requisição HTTP e manda para o service

auth.service.ts:

É o service de autenticação, onde fica a regra de negócio do cadastro, ele é quem processa os dados recebidos do controller

Figura 5 - DTO de cadastro de usuário

```
api > src > auth > dto > register.dto.ts > ...
1  import { IsEmail, IsNotEmpty, MinLength } from 'class-validator';
2
3  export class RegisterDto {
4    @IsNotEmpty()
5    name: string;
6
7    @IsEmail()
8    email: string;
9
10   @MinLength(6)
11   password: string;
12 }
13
```

Fonte: Elaborado pelos autores (2026)

Figura 6 - Controller de autenticação

```
api > src > auth > auth.controller.ts > ...
1  import { Body, Controller, Post } from '@nestjs/common';
2  import { AuthService } from '../auth.service';
3  import { RegisterDto } from '../dto/register.dto';
4
5  @Controller('auth')
6  export class AuthController {
7    constructor(private readonly authService: AuthService) {}
8
9    @Post('register')
10   async register(@Body() body: RegisterDto) {
11     return this.authService.register(body);
12   }
13 }
14
```

Fonte: Elaborado pelos autores(2026)

Figura 7 - Módulo de autenticação

```
api > src > auth > auth.module.ts > ...
1  import { Module } from '@nestjs/common';
2  import { AuthController } from './auth.controller';
3  import { AuthService } from './auth.service';
4  import { UsersModule } from '../users/users.module';
5
6  @Module({
7    imports: [UsersModule],
8    controllers: [AuthController],
9    providers: [AuthService],
10 })
11 export class AuthModule {}
12
```

Fonte: Elaborado pelos autores (2026)

Figura 8 - Entidade de usuário

```
api > src > users > user.entity.ts > User > ...
1  export class User {
2    id!: number;
3    name!: string;
4    email!: string;
5    password!: string;
6    createdAt!: Date;
7  }
```

Fonte: Elaborado pelos autores (2026)

Figura 9 - Entidade de usuário

```
api > src > auth > auth.service.ts > ...
1  import { Injectable } from '@nestjs/common';
2  import { UsersService } from '../users/users.service';
3  import { RegisterDto } from '../dto/register.dto';
4  import * as bcrypt from 'bcrypt';
5
6  @Injectable()
7  export class AuthService {
8    constructor(private usersService: UsersService) {}
9
10   async register(data: RegisterDto) {
11     const hashedPassword = await bcrypt.hash(data.password, 10);
12
13     const user = await this.usersService.create(
14       data.name,
15       data.email,
16       hashedPassword,
17     );
18
19     return {
20       id: user.id,
21       name: user.name,
22       email: user.email,
23       createdAt: user.createdAt,
24     };
25   }
26 }
```

Fonte: Elaborado pelos autores (2026)

PARTE 05 – ARQUITETURA, TECNOLOGIA E IMPLEMENTAÇÃO EVOLUTIVA DO SISTEMA

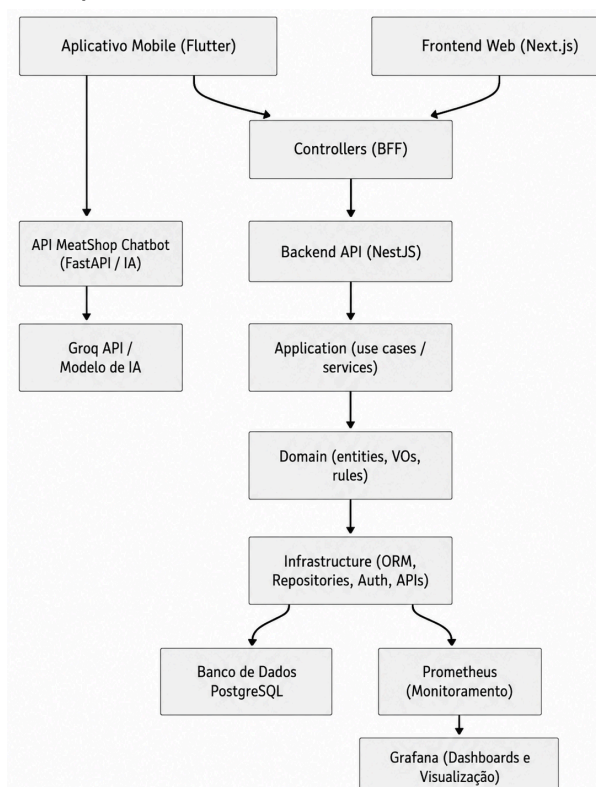
22 ARQUITETURA GERAL DO SISTEMA

O MeatShop foi estruturado seguindo uma arquitetura em camadas com o modelo cliente-servidor, com separação entre interface de usuário, processamento de regras de negócio, persistência de dados e monitoramento, esta organização permite maior escalabilidade, manutenção facilitada e independência entre os diferentes componentes da aplicação.

Dessa forma o projeto é composto por cinco principais partes: Frontend Web, Aplicativo Mobile, Backend API, Banco de Dados e Camada de Observabilidade, responsáveis respectivamente pela interação com o usuário, processamento das

operações do sistema, armazenamento das informações e monitoramento da infraestrutura. A comunicação entre as camadas ocorre principalmente por meio de requisições HTTP/REST, utilizando o formato JSON para troca de dados e tokens JWT para autenticação e autorização dos usuários.

Figura 10 - Diagrama da arquitetura



Fonte: Elaborado pelos autores (2026)

O diagrama acima demonstra a divisão da arquitetura onde primeiro temos o front web e mobile que são a camada de apresentação(cliente) onde o aplicativo mobile foi desenvolvido em Flutter e Web desenvolvido em Next.js são responsáveis por renderizar a interface do projeto e capturar as interações dos usuários além de gerenciar o estado da aplicação tanto web e mobile, como o frontend possui a responsabilidade da apresentação ele não acessa o banco diretamente ele envia requisições HTTP(get, post, put, delete) para o backend.

Após isso os controllers e requisições passam para o backend que é a camada de aplicação(servidor) que utiliza o framework NestJS do Node.js, sua

responsabilidade e centralizar todas as regras de negócio, segurança e processamento dos dados recebendo as requisições do frontend, validando permissões processando dados e devolvendo a resposta para o front além de possuir integração com prometheus para monitoramento e consulta de dados onde os mesmos são repassado ao grafana que gera dashboards e gráficos para visualização dos dados.

Além disso, também possuímos uma API de chatbot do Meatshop que é um serviço backend desenvolvido com o objetivo de fornecer respostas inteligentes baseadas em Inteligência Artificial, sendo especializada exclusivamente no domínio de carnes, cortes, receitas e técnicas de preparo culinário. A API foi desenvolvida em Python, utilizando o framework FastAPI, escolhido por sua alta performance, simplicidade de desenvolvimento e geração automática de documentação via Swagger.

E por fim temos o banco de dados que é a camada de dados onde sua responsabilidade é prover armazenamento seguro de todas as informações do sistema(usuários, configuração das integrações, dados do estoque do açougue, registro dos pedidos, renda e gastos do açougue) e apenas o backend tem acesso e permissão para ler ou gravar qualquer informação nesta camada.

23 TECNOLOGIAS E FERRAMENTAS DO PROJETO

Frontend Web (Next.js)

Interface gráfica voltada para administradores e funcionários da loja. O next permite criar aplicações web rápidas, escaláveis e otimizadas com menor complexidade de configuração assim permitindo gerenciamento de pedidos, controle de estoque, acompanhamento financeiro e cadastro de produtos e usuários, se comunicando com o Backend (NestJS) via requisições HTTP/REST para operações de leitura e escrita. Recebe dados formatados em JSON e exibe em componentes visuais (tabelas, cards, gráficos), enviando tokens JWT no cabeçalho das requisições para autenticação.

Backend API (NestJS)

Camada central da aplicação, responsável por processar requisições, aplicar regras de negócio, validar dados e interagir com o banco de dados. O backend é modularizado em diferentes domínios da aplicação, garantindo organização e escalabilidade.

Principais Módulos:

AuthModule: autenticação e autorização via JWT e refresh tokens.

UsersModule: cadastro, login e gerenciamento de perfis de usuários.

OrdersModule: criação e atualização de pedidos, controle de status e histórico.

ProductsModule: gerenciamento de estoque, categorias e preços.

FinanceModule: controle de despesas e receitas.

Recebe requisições dos clientes (Web e Mobile) e responde em JSON e realiza operações de leitura e gravação no PostgreSQL através do TypeORM. Expõe métricas no endpoint /metrics para o Prometheus e envia logs estruturados (via Winston) para monitoramento e auditoria.

Banco de Dados (PostgreSQL)

Armazenar e gerenciar todas as informações persistentes do sistema, como usuários, produtos, pedidos, estoque, histórico de transações e logs. É acessado exclusivamente pelo Backend (NestJS) por meio do TypeORM, que define as entidades e suas relações além de fornecer suporte a migrações versionadas, garantindo controle de alterações no esquema do banco comunicando-se internamente na rede Docker (meatshop-network), protegendo o acesso externo.

Prometheus (Monitoramento)

Coletar métricas do sistema em tempo real, como uso de CPU, memória, tempo de resposta das requisições e status de containers, consultando periodicamente o endpoint /metrics exposto pelo Backend NestJS. Armazena séries temporais de métricas para análise histórica e envia os dados para o Grafana, onde são transformados em dashboards.

Grafana (Dashboards e Visualização)

Exibir, em forma de painéis interativos, as métricas coletadas pelo Prometheus,

facilitando o acompanhamento de desempenho, erros e disponibilidade do sistema. Se conecta diretamente ao Prometheus para consumir dados de monitoramento e exibe dashboards personalizados com indicadores (KPIs) do backend e banco de dados podendo ser acessado via navegador web, permitindo análise visual pela equipe técnica.

Aplicativo Mobile (Flutter)

Aplicativo desenvolvido em Flutter voltado para o uso em smartphones Android e iOS, permitindo que clientes e entregadores interajam com o sistema de forma prática. Possibilita cadastro, login, acompanhamento de pedidos, notificações e visualização de descontos. Se comunica com o mesmo Backend (NestJS) via HTTP/REST, consumindo os mesmos endpoints que o frontend web e utiliza de autenticação baseada em JWT, garantindo segurança unificada entre plataformas. Sincroniza dados em tempo real quando conectado à internet, refletindo as alterações feitas no backend.

API Meatshop chatbot

As tecnologias utilizadas na API de chatbot incluem a linguagem Python para a construção do backend do chatbot, juntamente com o framework FastAPI, escolhido pela sua alta performance e pela geração automática de documentação disponibilizada automaticamente por meio do padrão OpenAPI/Swagger, facilitando testes e integração com outros componentes do sistema. A integração com o aplicativo mobile foi realizada utilizando Flutter, permitindo a comunicação entre o frontend e o serviço de Inteligência Artificial por meio de requisições HTTP e para a geração de respostas inteligentes, foi utilizada a Groq API, responsável por fornecer acesso a modelos avançados de linguagem natural. O serviço encontra-se hospedado na plataforma Render, que possibilita o deploy em nuvem e o acesso público via HTTPS.

24 IMPLEMENTAÇÃO POR CAMADAS DO SISTEMA

24.1 Frontend

No desenvolvimento do front-end da versão web (para açougues) do MeatShop, foi escolhida a linguagem TypeScript em vez de outras linguagens como JavaScript, Java ou Python, principalmente por causa da tipagem estática e da maior organização do código. O TypeScript adiciona tipos às variáveis, funções e objetos. Isso ajuda muito a evitar erros durante o desenvolvimento, porque muitos problemas podem ser identificados ainda na fase de programação, antes mesmo de executar o sistema. Em um sistema maior, como o MeatShop, que envolve várias funcionalidades (cadastro de usuários, pedidos, produtos, estoque, etc.), essa verificação ajuda a manter o código melhor desenvolvido.

Outro ponto importante é que facilita a organização e escalabilidade do projeto, já que ele traz uma estrutura de código mais clara e separada em módulos. Isso é útil principalmente onde várias partes do sistema precisam se comunicar, como interface, lógica de negócio e banco de dados. Além disso, o TypeScript é muito utilizado em frameworks de desenvolvimento web, como o Tailwind (que também é usado no nosso sistema) melhorando a produtividade do desenvolvimento do sistema.

A organização das telas e do fluxo do sistema do MeatShop foi pensada para deixar a navegação simples e intuitiva para o usuário, reduzindo a quantidade de etapas necessárias para realizar uma ação. Primeiro, as telas iniciais focam em login, cadastro e recuperação de senha, garantindo que o usuário consiga acessar o sistema. Depois disso, o fluxo principal leva o usuário diretamente para a visualização dos pedidos recentes, promoções ativas e um dashboard financeiro, que são umas das principais funcionalidades do sistema.

A partir dessa tela principal, o fluxo do sistema permite acessar facilmente outras áreas importantes, como gerenciamento de produtos, controle de estoque e acompanhamento de pedidos. Na parte de produtos, o açougue pode cadastrar novos itens, editar informações como nome, preço e descrição, além de atualizar a disponibilidade conforme o estoque. Essa organização facilita manter o catálogo sempre atualizado para os clientes.

Já na área de pedidos, o sistema mostra os pedidos realizados pelos clientes, permitindo que o açougue visualize os detalhes da compra e acompanhe o status de cada pedido. Assim, o estabelecimento consegue organizar melhor o preparo e

a separação dos produtos solicitados. Esse fluxo foi organizado dessa forma para que o açougueiro consiga gerenciar rapidamente os pedidos e os produtos, sem precisar navegar por muitas telas ou processos complexos. Dessa forma, o sistema ajuda a otimizar o tempo de trabalho dentro do açougue e melhora a organização das vendas realizadas pelo aplicativo.

24.2 Backend

O back-end do projeto foi desenvolvido com NestJS, seguindo uma arquitetura em camadas, na qual a aplicação permanece com suas responsabilidades organizadas em camadas específicas. De forma geral, a estrutura do código é composta por controllers, responsáveis por expor as rotas da API e receber as requisições, os services que concentram a maior parte das regras de negócio e entities do TypeORM, utilizadas para representar e salvar os dados no banco PostgreSQL. Essa organização facilita a manutenção, a evolução do sistema e a separação das responsabilidades entre as diferentes partes da aplicação. Atualmente, o projeto possui módulos e funcionalidades para autenticação, usuários, produtos, pedidos, promoções, dashboard, financeiro, métricas e integração com o Checkout Pro do Mercado Pago. Em relação aos requisitos funcionais, essa estrutura dá suporte principalmente aos fluxos de autenticação de usuários, gerenciamento e exibição de produtos, consulta e acompanhamento de pedidos, além da base para o processo de pagamento online. Para a segurança, o sistema utiliza JWT com refresh token para controle de autenticação e renovação de sessão, além de bcrypt para gerar o hash das senhas antes do salvamento no banco de dados. Além disso, o back-end também foi desenvolvido com recursos de apoio à operação e ao monitoramento da aplicação. Para isso, o projeto utiliza Winston no registro de logs e Prometheus/Grafana no acompanhamento de métricas e visualização de dados de observabilidade, o projeto também possui um workflow de integração contínua com o GitHub Actions, responsável por executar verificações de qualidade, como lint, testes

E2E e validação de build com o Docker, além de gerar artefatos em .zip ao final da execução para garantir a qualidade durante o desenvolvimento.

24.3 Integração

A comunicação entre as camadas do sistema MeatShop ocorre seguindo uma arquitetura em camadas composta pelo frontend, backend e banco de dados, onde a interação começa quando o usuário acessa o sistema pela interface web desenvolvida com Next.js e TypeScript, que é responsável por apresentar as telas, coletar informações digitadas pelo usuário e enviar requisições HTTP para a API do sistema, esta camada cuida apenas da interface e da experiência do usuário, não acessando diretamente o banco de dados.

Quando uma ação é realizada, como login, cadastro de produtos ou consulta de pedidos, o frontend envia uma requisição para o backend desenvolvido em NestJS. No backend, a requisição é recebida pelos controllers, que são responsáveis por expor as rotas da API e encaminhar os dados para os services, onde ficam concentradas as regras de negócio do sistema, nessa etapa são realizadas validações importantes, como verificação de permissões do usuário, validação de dados e aplicação das regras do sistema. Após esse processamento, os services utilizam as entidades do TypeORM para acessar e manipular os dados no banco de dados PostgreSQL.

O banco de dados armazena todas as informações estruturadas do sistema, como usuários, unidades, produtos, estoque, pedidos e pagamentos. Sempre que o backend precisa consultar, inserir ou atualizar dados, ele realiza essas operações por meio dessas entidades, garantindo que todas as regras de integridade e relacionamento entre as tabelas sejam respeitadas. Após a execução da operação no banco, o backend retorna uma resposta para o frontend, que então atualiza a interface e apresenta o resultado para o usuário.

Entre os principais fluxos do sistema estão o processo de autenticação, no qual o usuário realiza login e recebe um token JWT para acessar as funcionalidades protegidas; o gerenciamento de produtos e estoque, que permite ao açougue cadastrar e atualizar itens disponíveis para venda; e o fluxo de pedidos, onde as compras realizadas pelos clientes são registradas, processadas e acompanhadas

pelo estabelecimento. Durante esse processo, o sistema também pode integrar pagamentos via Mercado Pago, atualizar o status do pedido e registrar todas as informações no banco de dados.

Dessa forma, o MeatShop mantém uma comunicação organizada entre suas camadas, garantindo que as interfaces lidem apenas com a interação do usuário, o backend concentre a lógica do sistema e o banco de dados seja responsável pela persistência segura das informações. Essa separação facilita a manutenção, a escalabilidade e a confiabilidade do sistema ao longo do seu desenvolvimento.

O deploy em ambiente de produção ainda não foi realizado porque nosso foco nesta etapa foi a entrega funcional, validação acadêmica, testes locais e estruturação da aplicação. Além disso, antes de publicar o sistema em nuvem, devemos concluir uma etapa de preparação mais segura, envolvendo homologação, configuração de domínio, certificados HTTPS, variáveis sensíveis e banco de dados gerenciado. A previsão para o deploy é para o final de setembro de 2026. Até essa data, o objetivo é finalizar os ajustes de ambiente, revisar os requisitos de segurança, validar os testes finais e preparar a infraestrutura em nuvem para receber a aplicação de forma estável e segura. Utilizamos a pipeline no GitHub Actions para automatizar algumas etapas do processo, como execução de testes, build da aplicação, validação do código e preparação para envio das imagens Docker. Como o projeto já possui Dockerfile e Docker Compose no ambiente local, para produção iremos realizar o deploy em nuvem utilizando containers em serviços como AWS ou Azure. Na AWS, as imagens do backend e frontend seriam enviadas para o Amazon ECR e executadas no ECS com Fargate. O banco PostgreSQL deixaria de rodar em container e passaria para um serviço gerenciado, como o Amazon RDS para garantir mais segurança, backup e disponibilidade. As variáveis sensíveis, como senha do banco, JWT secret e chaves do Mercado Pago seriam armazenadas em um serviço seguro, como AWS Secrets Manager ou Azure Key Vault, evitando o uso de arquivos .env em produção. O acesso à aplicação seria feito por um Load Balancer com HTTPS, enquanto serviços internos, como banco, métricas e monitoramento ficariam protegidos em rede privada.

Checklist de pré-requisitos:

1. Validar os testes automatizados no GitHub Actions
2. Confirmar o build do backend e frontend sem erros
3. Criar as imagens Docker de produção
4. Configurar o ambiente em nuvem (AWS ou Azure)
5. Publicar as imagens em um registry, como Amazon ECR ou Azure Container Registry
6. Configurar banco PostgreSQL gerenciado
7. Migrar e validar as tabelas do banco de dados em produção
8. Configurar variáveis sensíveis em Secrets Manager ou Key Vault
9. Configurar domínio e certificado HTTPS
10. Configurar Load Balancer
11. Proteger banco de dados, métricas e serviços internos em rede privada
12. Validar integração com Mercado Pago em ambiente seguro
13. Realizar testes finais de autenticação, cadastro, pedidos, pagamentos e gestão financeira
14. Configurar logs, métricas e monitoramento
15. Definir rotina de backup e recuperação do banco de dados
16. Criar plano de rollback para caso ocorra falha após a publicação.

25 ORGANIZAÇÃO DO CÓDIGO E REPOSITÓRIO

Mobile: A principal organização da aplicação fica concentrada dentro da pasta lib, onde o código-fonte será dividido por responsabilidade. A pasta models será utilizada para representar as estruturas de dados da aplicação, como usuários, produtos, pedidos e demais entidades do sistema. A pasta providers ficará responsável pelo gerenciamento de estado e pela lógica de controle dos dados exibidos em tela, intermediando a comunicação entre a interface e as regras da aplicação. Já a pasta ui concentra a camada visual do sistema, sendo subdividida em screens, para as telas principais, widgets, para elementos reutilizáveis

menores, e components, para componentes de interface mais estruturados e reaproveitados em diferentes partes do aplicativo.

Essa divisão busca garantir maior clareza, reaproveitamento de código e facilidade de manutenção, evitando que regras de negócio, estrutura de dados e interface fiquem misturadas. Com isso, o projeto segue uma organização coerente com a arquitetura proposta, permitindo que novas funcionalidades sejam adicionadas de forma progressiva e controlada. Além disso, o repositório também demonstra evolução contínua por meio da separação em commits temáticos, documentação inicial e estruturação das primeiras telas e fluxos da aplicação, reforçando o cuidado do grupo com padronização, legibilidade e crescimento sustentável do código.

Web: Até o momento a organização principal do código está distribuída em pastas como app, components, hooks, lib e public. A pasta app concentra as rotas, páginas e estrutura de navegação da aplicação web, seguindo o padrão do Next.js. A pasta components armazena os componentes visuais reutilizáveis, como cards, formulários, tabelas e elementos de interface compartilhados entre diferentes telas. Já hooks reúne lógicas reutilizáveis relacionadas ao comportamento da interface, enquanto lib concentra funções auxiliares, integrações e utilitários de apoio ao funcionamento da aplicação. A pasta public é destinada aos arquivos estáticos, como imagens, ícones e demais recursos públicos utilizados no sistema.

Essa estrutura foi adotada para manter a interface organizada, facilitar a reutilização de código e tornar o desenvolvimento mais sustentável ao longo da evolução do projeto. A separação entre páginas, componentes, funções auxiliares e arquivos estáticos reduz o acoplamento, melhora a legibilidade e contribui para a manutenção do sistema. Além disso, o histórico do repositório demonstra evolução contínua por meio de ajustes funcionais, correções e integração progressiva entre frontend e backend, reforçando a preocupação do grupo com organização, versionamento e crescimento estruturado da aplicação.

Backend: O projeto foi estruturado para manter a separação entre configuração da aplicação, módulos de negócio, entidades de persistência e recursos de suporte técnico. A organização do código evidencia uma preocupação com modularidade e manutenção, concentrando a lógica principal em arquivos e pastas bem definidos.

Entre os diretórios existentes estão auth, finance, metrics, config, entities, migrations, seed e common/logger, além de arquivos centrais como main.ts, app.module.ts, app.controller.ts e app.service.ts, que compõem a base inicial da aplicação em NestJS.

A pasta auth reúne a lógica de autenticação e controle de acesso, enquanto finance organiza responsabilidades relacionadas ao módulo financeiro. A pasta metrics concentra a exposição de métricas e integração com monitoramento, e config centraliza configurações gerais do sistema. Já entities define as estruturas persistidas no banco de dados, representando os principais elementos do domínio da aplicação, como usuários, produtos, pedidos e pagamentos. As pastas migrations e seed são utilizadas, respectivamente, para versionamento da estrutura do banco e inserção de dados iniciais de apoio ao ambiente. Além disso, a pasta common/logger concentra recursos compartilhados de registro de logs, contribuindo para auditoria e rastreabilidade das operações.

Também é possível observar a presença de controllers e DTOs específicos, como arquivos de produtos, pedidos, dashboard e usuários, mostrando que o backend já segue uma separação entre entrada de dados, validação e processamento das regras da aplicação. Essa organização contribui para que as responsabilidades não fiquem misturadas, facilitando testes, manutenção e evolução do sistema. Dessa forma, o repositório backend demonstra coerência com a arquitetura proposta no projeto, permitindo que novos módulos sejam adicionados de forma progressiva, sem comprometer a clareza estrutural do código.

PARTE 06 – QUALIDADE, SEGURANÇA, AVALIAÇÃO E EXTENSÕES DO PROJETO

27 QUALIDADE DO SOFTWARE E TESTES

O ambiente de execução dos testes foi configurado na máquina local através do visual studio com o terminal de comandos Powershell integrado para execução dos comandos e sendo sua maioria com exceção do teste de cadastro de produto realizados com o grafana k6 pelo fato de que o grafana já estar sendo utilizado no

projeto para monitoramento e análise de dados em tempo real do projeto e como possuía a ferramenta grafana k6 que é focada em testes simulando usuários reais de diversos tipos de carga para obter as métricas, o projeto estava sendo executado via Docker aonde a api ficava disponível para receber as requisições dos testes de carga e stress com mocks para o teste unitário onde eram inseridos produtos no sistema e por fim os testes de performance sendo tempo médio de resposta, p95/p99, taxa de erro e throughput(requisições por segundo). Realizamos simulações de testes de carga e pico para simular como seria a resposta do sistema a grandes picos de uso ambos se relacionam com as RNF 003 e 005, onde para o teste de carga foi simulado 50 usuários ativos durante 2 minutos onde 5400 requisições foram feitas e atendidas rapidamente e sem problemas para demonstrar como seria a resposta da aplicação ao funcionando normalmente, abaixo uma imagem com dados demonstrando pelo k6 uma extensão do grafana que retorna os dados dos testes

Figura 11 - Teste de carga

```
PS C:\Users\samue\Documents\Projeto Integrador\Projeto-Integrador-MeatShop-main\MeatShop - PT\meatshop-performance-tests> k6 run pedidos-load-test.js
>>

  Grafana

execution: local
script: pedidos-load-test.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 2m30s max duration (incl. graceful stop):
  * default: Up to 50 looping VUs for 2m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

TOTAL RESULTS

checks_total.....: 5400    44.821663/s
checks_succeeded...: 100.00% 5400 out of 5400
checks_failed.....: 0.00%   0 out of 5400

✓ status é 201 (pedido criado)
✓ resposta menor que 1s

HTTP
http_req_duration.....: avg=2.69ms min=0s med=2.64ms max=45.56ms p(90)=4.23ms p(95)=4.64ms
  ( expected response:true )...: avg=2.69ms min=0s med=2.64ms max=45.56ms p(90)=4.23ms p(95)=4.64ms
http_req_failed.....: 0.00%   0 out of 2700
http_reqs.....: 2700    22.410832/s

EXECUTION
iteration_duration.....: avg=1s    min=1s med=1s    max=1.04s p(90)=1s p(95)=1s
iterations.....: 2700    22.410832/s
vus.....: 1    min=1    max=50
vus_max.....: 50    min=50    max=50
```

Fonte: Elaborado pelos autores (2026)

No caso do teste de pico ao invés de 50 usuários que entrariam na aplicação ao longo do tempo e se manteriam durante 2 minutos simulamos a entrada subida de 500 usuários de uma vez servindo para demonstrar situações de pico como em grandes promoções nos açougues aumentando a demanda da aplicação além de ser observado um aumento no tempo de resposta porém a aplicação trava e consegue atender as requisições, abaixo a imagem com os resultados do teste.

Figura 12 - Teste de pico de usuários

```
PS C:\Users\samue\Documents\Projeto Integrador\Projeto Integrador-MeatShop-main\MeatShop - PI\meatshop-performance-tests> k6 run pedidos-spike-test.js
>>

  Grafana

execution: local
script: pedidos-spike-test.js
output: -

scenarios: (100.00%) 1 scenario, 500 max VUs, 1m30s max duration (incl. graceful stop):
  * default: Up to 500 looping VUs for 1m0s over 5 stages (gracefulRampDown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'p(95)<2000' p(95)=572.31ms

http_req_failed
✓ 'rate<0.05' rate=0.00%

TOTAL RESULTS

checks_total.....: 116238 1937.248763/s
checks_succeeded...: 100.00% 116238 out of 116238
checks_failed.....: 0.00% 0 out of 116238

✓ status é 201 (pedido criado)
✓ resposta menor que 2s

HTTP
http_req_duration.....: avg=181.96ms min=0s med=17.58ms max=1.11s p(90)=511.85ms p(95)=572.31ms
  { expected_response:true }...: avg=181.96ms min=0s med=17.58ms max=1.11s p(90)=511.85ms p(95)=572.31ms
http_req_failed.....: 0.00% 0 out of 58119
http_reqs.....: 58119 968.624381/s
```

Fonte: Elaborado pelos autores (2026)

Fase 2 - 7º período

Também realizamos testes com mocks a fim de avaliar o cadastro de produtos que possui diversas regras para o cadastro, então testamos 6 cenários diferentes onde o 1º segue todas as regras e os 5 próximos desobedecem alguma delas.

Nesta imagem simulamos o cenário em que o cadastro é feito corretamente:

Figura 13 - Teste de cadastro corretamente (dados mocks)

```
describe('Fluxo de Cadastro de Produto', () => {
  beforeEach(() => {
    save: jest.fn()
  });

  controller = new ProductsController(mockProductRepo);

  // 1 - Produto válido
  it('deve cadastrar produto válido', async () => {
    const dto = {
      status: 'ACTIVE' as const,
```

Fonte: Elaborado pelos autores (2026)

e este a caso ele tente cadastrar sem colocar o nome do produto:

Figura 14 - Teste de cadastro de maneira incorreta(dados mocks)

```
// 2 - Produto sem nome
it('não deve permitir cadastro sem nome', async () => {

  const dto = {
    status: 'ACTIVE' as const,
    name: '',
    category: 'Bovina',
    cut: 'Nobre',
    description: 'Carne premium',
    quantity: "10 kg",
    price: 50
  };

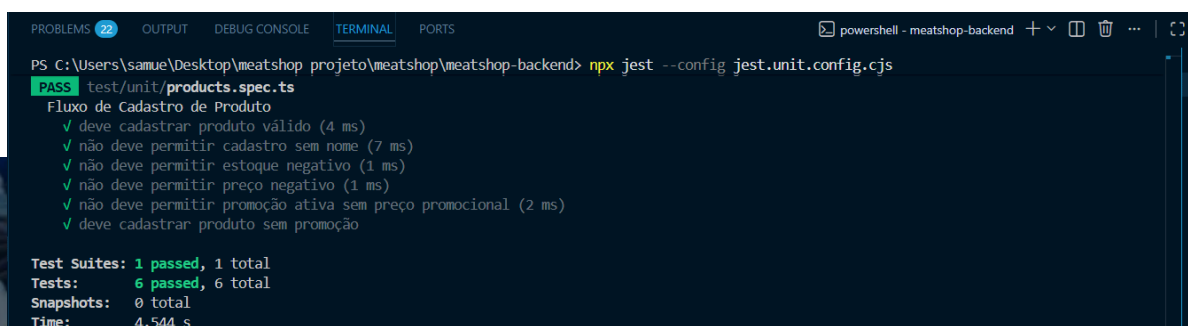
  mockProductRepo.save.mockRejectedValue(new Error('Nome obrigatório'));

  await expect(controller.create(dto))
    .rejects
    .toThrow('Nome obrigatório');
```

Fonte: Elaborado pelos autores (2026)

E aqui os resultados esperados no terminal demonstrando que todos os cenários passaram no teste onde o primeiro está sendo realizado o cadastro correto e os outros cenários deram falha porque os produtos devem ser cadastrados com todas as informações válidas ou seja sem faltar dados do produto ou dados irreais como preço e estoque negativo.

Figura 15 - Resultado de ambos os testes(dados mocks)



```
PS C:\Users\samue\Desktop\meatshop\projeto\meatshop\meatshop-backend> npx jest --config jest.unit.config.cjs
PASS test/unit/products.spec.ts
Fluxo de Cadastro de Produto
  ✓ deve cadastrar produto válido (4 ms)
  ✓ não deve permitir cadastro sem nome (7 ms)
  ✓ não deve permitir estoque negativo (1 ms)
  ✓ não deve permitir preço negativo (1 ms)
  ✓ não deve permitir promoção ativa sem preço promocional (2 ms)
  ✓ deve cadastrar produto sem promoção

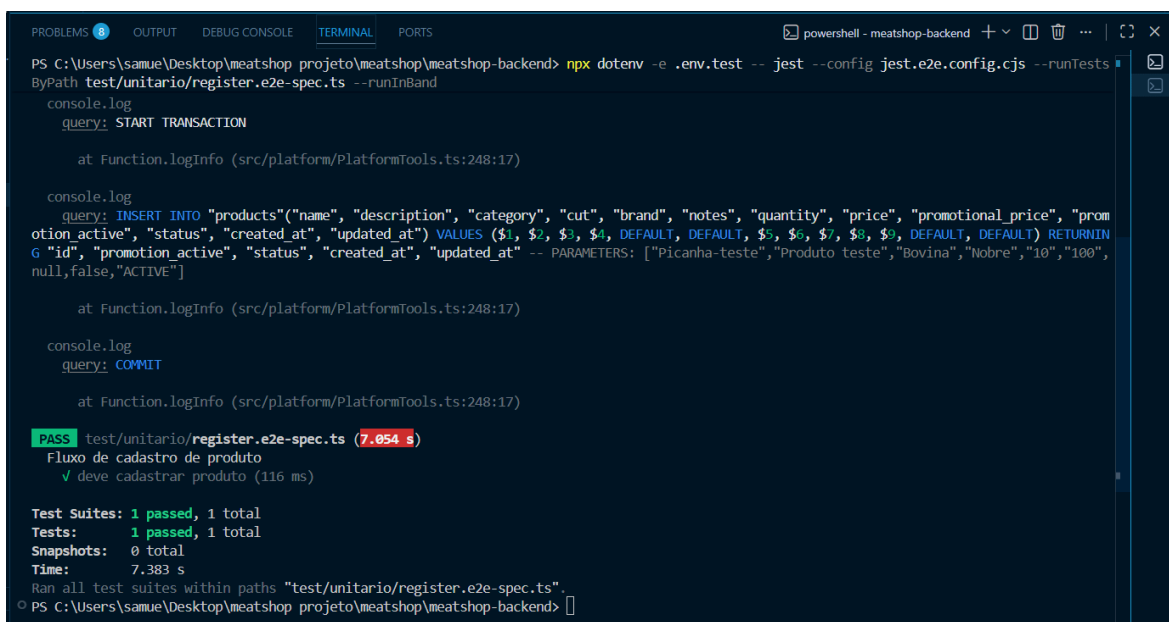
Test Suites: 1 passed, 1 total
Tests: 6 passed, 6 total
Snapshots: 0 total
Time: 4.544 s
```

Fonte: Elaborado pelos autores (2026)

Fase 3 - 7º período

Nesta fase aplicamos o teste acima porém sem uso de mocks usando o banco real do sistema a fim de garantir que o sistema só pode salvar produtos com todos os dados corretos sem falta nenhum ou com uma informação errada como valor ou quantidade negativa abaixo o que acontece com o cadastro correto do produto

Figura 16 - Teste de cadastro real



```
PS C:\Users\samue\Desktop\meatshop projeto\meatshop\meatshop-backend> npx dotenv -e .env.test -- jest --config jest.e2e.config.cjs --runTestsByPath test/unitario/register.e2e-spec.ts --runInBand
console.log
  query: START TRANSACTION

    at Function.logInfo (src/platform/PlatformTools.ts:248:17)

console.log
  query: INSERT INTO "products"("name", "description", "category", "cut", "brand", "notes", "quantity", "price", "promotional_price", "promotion_active", "status", "created_at", "updated_at") VALUES ($1, $2, $3, $4, DEFAULT, DEFAULT, $5, $6, $7, $8, $9, DEFAULT, DEFAULT) RETURNING "id", "promotion_active", "status", "created_at", "updated_at" -- PARAMETERS: ["Picanha-teste","Produto teste","Bovina","Nobre","10","100",null,false,"ACTIVE"]

    at Function.logInfo (src/platform/PlatformTools.ts:248:17)

console.log
  query: COMMIT

    at Function.logInfo (src/platform/PlatformTools.ts:248:17)

PASS test/unitario/register.e2e-spec.ts (7.054 s)
Fluxo de cadastro de produto
  ✓ deve cadastrar produto (116 ms)

Test Suites: 1 passed, 1 total
Tests: 1 passed, 1 total
Snapshots: 0 total
Time: 7.383 s
Ran all test suites within paths "test/unitario/register.e2e-spec.ts".
PS C:\Users\samue\Desktop\meatshop projeto\meatshop\meatshop-backend>
```

Fonte: Elaborado pelos autores (2026)

Os testes de performance foram executados em conjunto onde foi simulado até 30 usuários virtuais durante 2 minutos, buscávamos avaliar 4 métricas o tempo médio

de resposta onde este tempo não passou do 9,66ms abaixo do limite de 1000ms que foi definido, os percentis 95/99 o P95 foi de 26,7ms, indicando que 95% das requisições responderam em até esse tempo e o percentil P99 foi de 51,13ms, demonstrando que mesmo nos casos mais lentos a API manteve boa performance, já a taxa de erro foi de 0,00% aonde a métrica estabelecida foi menor que 5% e por fim tivemos uma taxa de 14 requisições por segundo superando a taxa de 5 requisições por segundo e ao final tivemos 1780 requisições com 2 checks que validam se aplicação realmente funciona ou seja se as requisições foram atendidas e retornaram status 200 dessa forma todas as requisições passaram com sucesso neste caso como é uma simulação os resultados tendem a ser melhores e ter um desempenho excelente porém novos testes serão feitos no ambiente de produção para que o projeto seja avaliado de forma mais clara para identificação de limitações, gargalos, interações externas e diversas outras variáveis que podem vir a interferir na aplicação.

Figura 17: Demonstrando os resultados obtidos nos 4 testes

```
PS C:\Users\samue\Desktop\meatshop_projeto\meatshop\meatshop-backend> k6 run \test\unitario\performance_test.ts

Grafana

execution: local
script: \test\unitario\performance_test.ts
output: -

scenarios: (100.00%) 1 scenario, 30 max VUs, 2m30s max duration (incl. graceful stop):
  * default: Up to 30 looping VUs for 2m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'avg<1000' avg=9.66ms
✓ 'p(95)<1500' p(95)=26.7ms
✓ 'p(99)<2500' p(99)=51.13ms

http_req_failed
✓ 'rate<0.05' rate=0.00%

http_reqs
✓ 'rate>5' rate=14.807351/s
```

Fonte: Elaborado pelos autores (2026)

Figura 18 - Resultados dos checks e informações mais profundas sobre cada teste


```
TOTAL RESULTS
checks_total.....: 3560    29.614701/s
checks_succeeded....: 100.00% 3560 out of 3560
checks_failed.....: 0.00%   0 out of 3560

✓ status é 200
✓ tempo menor que 2s

HTTP
http_req_duration.....: avg=9.66ms min=660.3µs med=6.82ms max=137.65ms p(90)=16.78ms p(95)=26.7ms
{ expected_response:true }...: avg=9.66ms min=660.3µs med=6.82ms max=137.65ms p(90)=16.78ms p(95)=26.7ms
http_req_failed.....: 0.00%   0 out of 1780
http_reqs.....: 1780    14.807351/s

EXECUTION
iteration_duration.....: avg=1.01s min=1s      med=1s      max=1.13s    p(90)=1.01s  p(95)=1.02s
iterations.....: 1780    14.807351/s
vus.....: 1      min=1      max=30
vus_max.....: 30    min=30     max=30

NETWORK
data_received.....: 5.4 MB 45 kB/s
data_sent.....: 139 kB 1.2 kB/s

running (2m00.2s), 00/30 VUs, 1780 complete and 0 interrupted iterations
default ✓ [=====] 00/30 VUs  2m0s
```

Fonte: Elaborado pelos autores (2026)

Quanto ao fluxo de pagamentos e entrega estão ligados não só com a web mais com o mobile que está em desenvolvimento, por isso não foram colocados no escopo de testes já que ainda não possui a integração com o back-end dessa forma os testes envolvendo estas funcionalidades serão feitos posteriormente assim que o front-end mobile estiver devidamente finalizado e integrado com o back-end.

28 SEGURANÇA DA INFORMAÇÃO

Utilizando de tokens para garantir a segurança de acesso ao sistema quando o usuário realiza o login que o back válida com os dados do banco o que gera o token com id, permissões e tempo de expiração que permite acesso com as devidas limitações do tipo de usuário ao sistema e caso o usuário não utilize o token após um período ele expira e não pode ser mais utilizado para acessar o sistema garantindo o acesso apenas aqueles que criarem conta tanto clientes quanto para o açougue.

Para garantir a segurança da integração com serviços externos da IA, foram adotadas algumas medidas sendo o uso de variáveis de ambiente onde a chave da API da Groq é armazenada de forma segura no ambiente do servidor, sem ser

exposta no código-fonte, o serviço também opera sobre HTTPS, garantindo comunicação segura entre cliente e servidor onde a IA está sendo treinada e validada apenas para serviços de auxiliá para tipos de cortes melhores carnes e receitas culinárias sendo incapaz de ser usada fora do escopo do sistema.

Além disso utilizarmos a integração com o banco de dados firebase que por si já possui uma criptografia que funciona automaticamente em duas camadas principais sendo criptografia em repouso (at rest), protegendo dados armazenados nos servidores do Google, e criptografia em trânsito (in transit), usando TLS para proteger dados na internet, também implementamos um sistema de restrição de senha onde as senhas tanto da parte web quanto mobile deve ter 8 dígitos incluindo números, caractere especial e letras maiúscula e minúscula, dessa forma dificultando as chances de ataques usando geradores de senha para descobrir a senha dos usuários e garantindo a segurança dos dados que sobem ao banco como dados pessoais e informações do açougue.

Devido aos tipos de dados que o nosso projeto deve tratar, como e-mail, CPF/CNPJ, endereços e etc, nós consideramos os princípios da LGPD, como finalidade, necessidade, segurança e prevenção, utilizando essas informações apenas para cadastro, autenticação, controle de pedidos, gestão financeira e acompanhamento de pagamentos. Também prevemos o uso do consentimento explícito do usuário, o usuário deverá aceitar os termos de uso e a política de privacidade para utilização do aplicativo. A política de privacidade deve informar de forma clara quais dados são coletados, para quais finalidades eles são utilizados, como são protegidos e quais são os direitos dos titulares. Para proteger esses dados, as senhas são armazenadas por meio de hash com bcrypt, evitando que fiquem salvas em texto puro. A autenticação utiliza JWT com access token e refresh token em cookies HttpOnly, reduzindo riscos de acesso indevido. O sistema também máscara informações sensíveis nos logs, como senhas, tokens e CPF/CNPJ. Para pagamentos, utilizamos o Checkout Pro do Mercado Pago, garantindo mais uma camada de proteção fornecida pelo próprio Mercado Pago e não armazenamos dados completos de cartão, CVV ou informações bancárias. São salvos apenas dados operacionais, como status da transação, valor pago e identificadores de pagamento. Em relação à retenção de dados, o sistema irá

manter as informações apenas pelo tempo necessário para cumprir as necessidades, obrigações legais, controle financeiro e histórico de pedidos. Após esse período, os dados poderão ser excluídos, anonimizados ou mantidos somente quando houver justificativa legal ou operacional. Para respeitar os direitos dos titulares dos dados, devemos permitir que o usuário possa solicitar acesso, correção, exclusão, portabilidade ou informações sobre o uso de seus dados pessoais. Também iremos definir protocolos para caso ocorra um incidente de segurança, como vazamento, acesso indevido ou exposição de dados. Nesse processo devemos incluir a identificação do problema, contenção do incidente, análise do impacto, comunicação aos responsáveis e, quando necessário, notificação aos titulares dos dados e à Autoridade Nacional de Proteção de Dados.

As subseções 29 e 30 ainda serão construídas no próximo período seguindo as divisões por período instruídas para a documentação.

31 REFERÊNCIAS

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software Architecture in Practice. 4. ed. Boston: Addison-Wesley, 2021.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 22 jun. 2026.

ISO/IEC/IEEE. ISO/IEC/IEEE 29148:2018: Systems and software engineering — Life cycle processes — Requirements engineering. Geneva: ISO, 2018.

LAUDON, Kenneth C.; TRAYER, Carol Guercio. E-commerce 2023–2024: business, technology, society. 18. ed. Harlow: Pearson, 2024.

NIELSEN NORMAN GROUP. What is User Experience? 2024. Disponível em: <https://www.nngroup.com/articles/what-is-user-experience/>. Acesso em: 22 jun. 2026.

OWASP FOUNDATION. OWASP Top 10:2025. 2025. Disponível em: <https://owasp.org/Top10/2025/en/>. Acesso em: 22 jun. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

W3C. Web Content Accessibility Guidelines (WCAG) 2.2. 2024. Disponível em: <https://www.w3.org/TR/WCAG22/>. Acesso em: 22 jun. 2026.

ALMEIDA, Fernando de Souza et al. Transformação digital e a quarta revolução industrial: teoria da inovação disruptiva. Revista PMKT, 2021. Disponível em: https://revistapmkt.com.br/wp-content/uploads/2021/12/1-Transformac_o-Digital-e-a-Quarta-Revoluc_o-Industrial-Teoria-da-Inovac_o-Disruptiva-ENSAIO.pdf. Acesso em: 21 jun. 2026.

MENDES, Luciano Ferreira et al. E-commerce: O comércio eletrônico: uma perspectiva de sucesso para as organizações da sociedade da informação. Revista Eletrônica de Administração, 2021. Disponível em: <https://www.periodicosibepes.org.br/index.php/recadm/article/view/427/325>. Acesso em: 21 jun. 2026.

NORMAN, Donald A. O design do dia a dia. Disponível em: <https://brasil.uxdesign.cc/o-design-do-dia-a-dia-donald-a-norman-bce97ee1f032>. Acesso em: 21 jun. 2026.

ALURA. Padrões arquiteturais: arquitetura de software descomplicada. Disponível em: <https://www.alura.com.br/artigos/padroes-arquiteturais-arquitetura-software-descomplicada>. Acesso em: 21 jun. 2026.